

Agent, Evaluation and RL Verifier



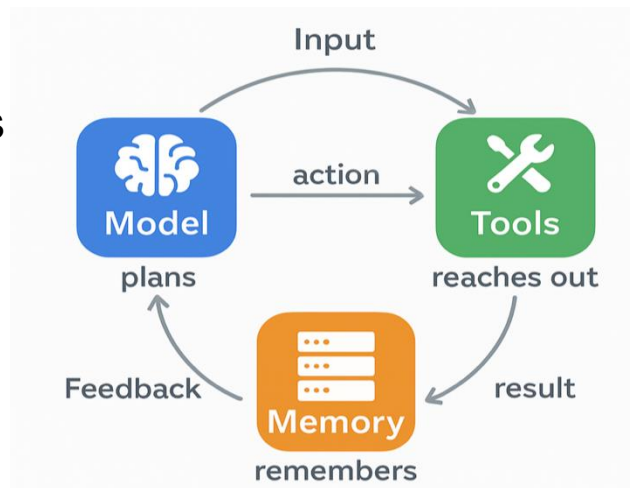
Che Guan, Ph.D.
April 2026

Section 1:

LLM Agent

Agent Components

- An autonomous agent is typically composed of 4 main components:
 - **The Planner:** The LLM decomposes a high-level goal into smaller, actionable sub-tasks.
 - **Memory:** Short-term *and* Long-term Memories
 - **Tools:** The set of APIs or functions the agent can call.
 - **The Loop:** Unlike a single prompt, an agent runs in a loop: Observe → Think → Act → Observe.



Section 1.1:

Agent

ReAct

- A ReAct agent uses the “reasoning and acting” framework to combine CoT reasoning with external tool use.

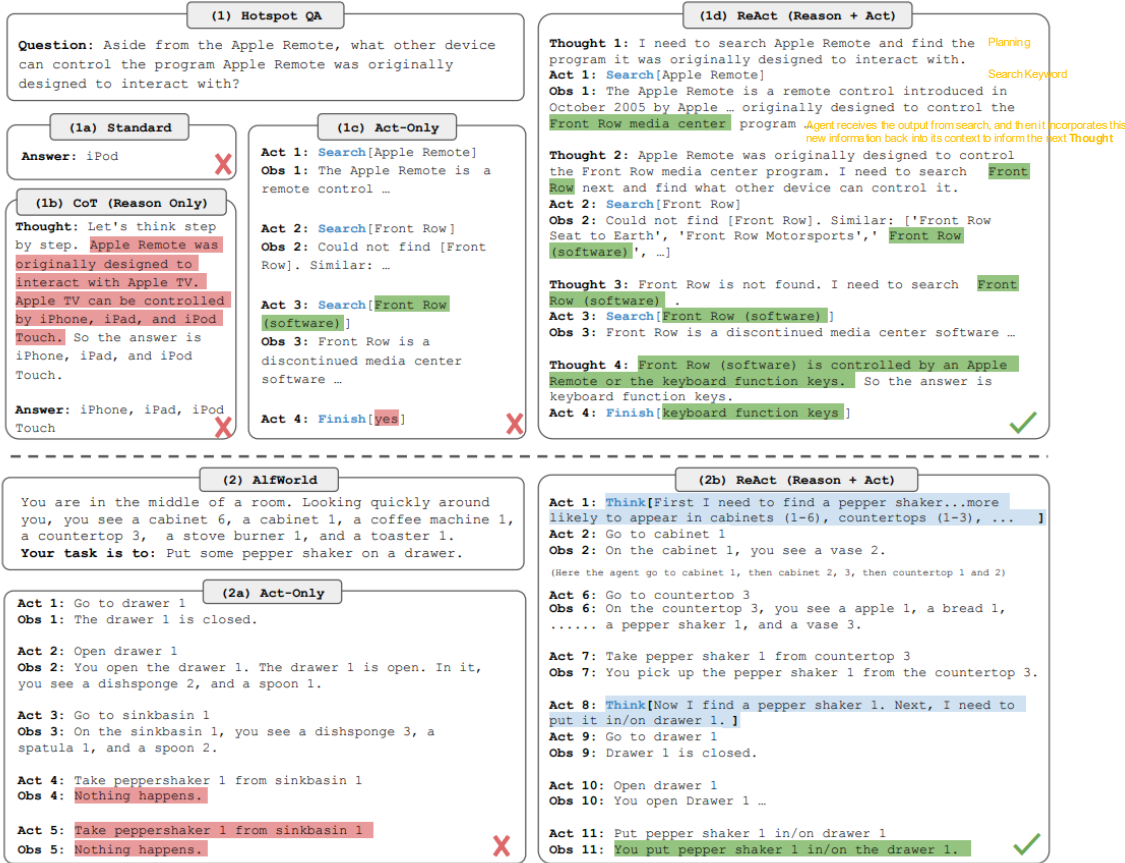
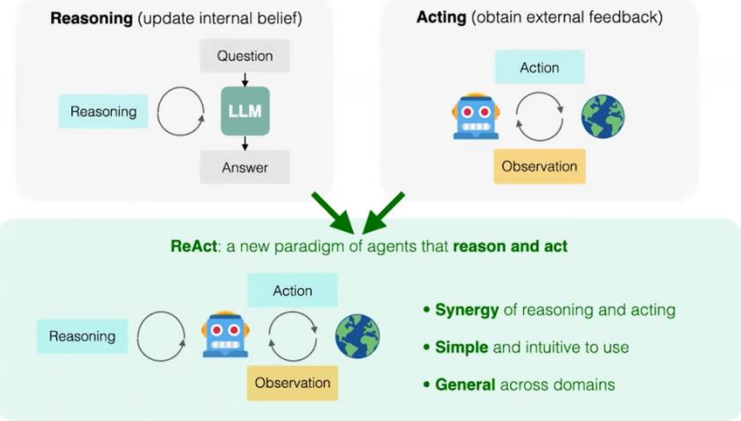
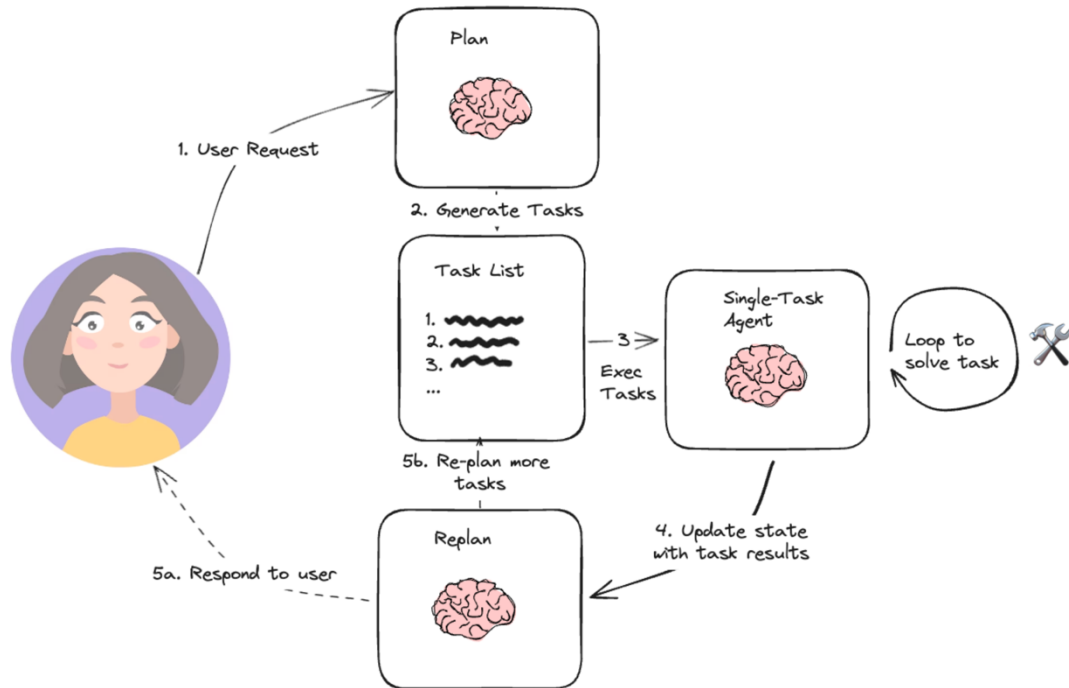


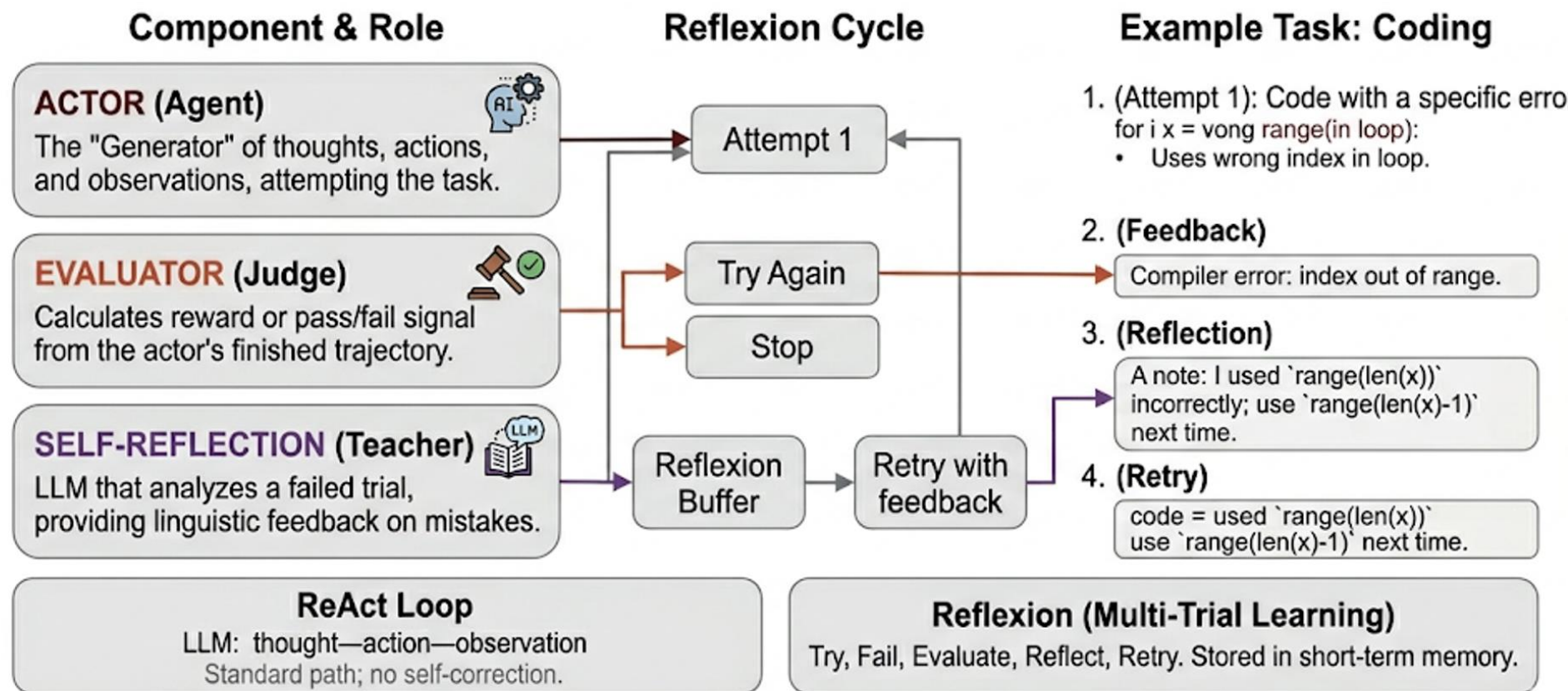
Figure 1: (1) Comparison of 4 prompting methods, (a) Standard, (b) Chain-of-thought (CoT, Reason Only), (c) Act-only, and (d) ReAct (Reason+Act), solving a HotpotQA (Yang et al., 2018) question. (2) Comparison of (a) Act-only and (b) ReAct prompting to solve an AlfWorld (Shridhar et al., 2020b) game. In both domains, we omit in-context examples in the prompt, and only show task solving trajectories generated by the model (Act, Thought) and the environment (Obs).

Plan-and-Execute Agents

- A planner, which prompts an LLM to generate a multi-step plan to complete a large task.
- Executor(s), which accept the user query and a step in the plan and invoke 1 or more tools to complete that task.



Understanding the Reflexion Agentic Framework



Reflexion: Language Agents with Verbal Reinforcement Learning

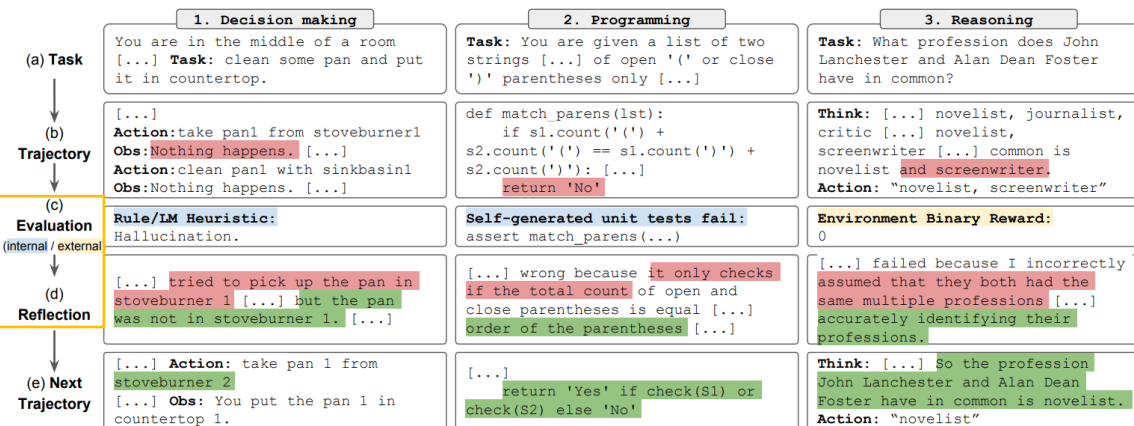
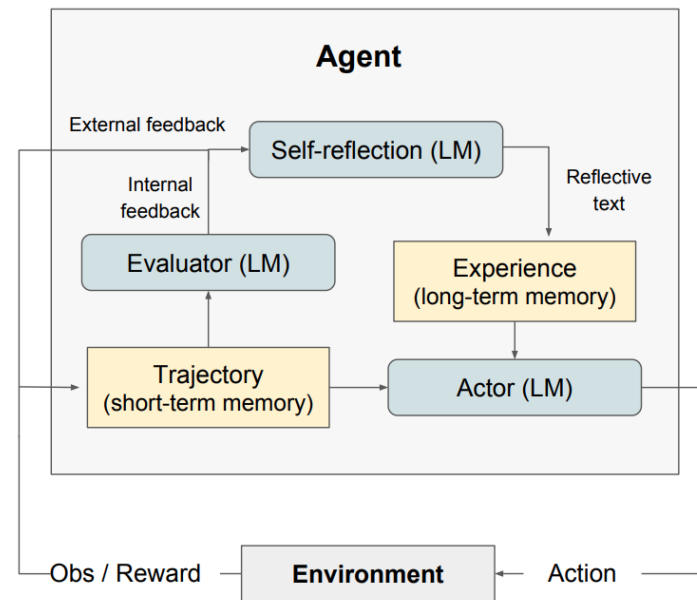
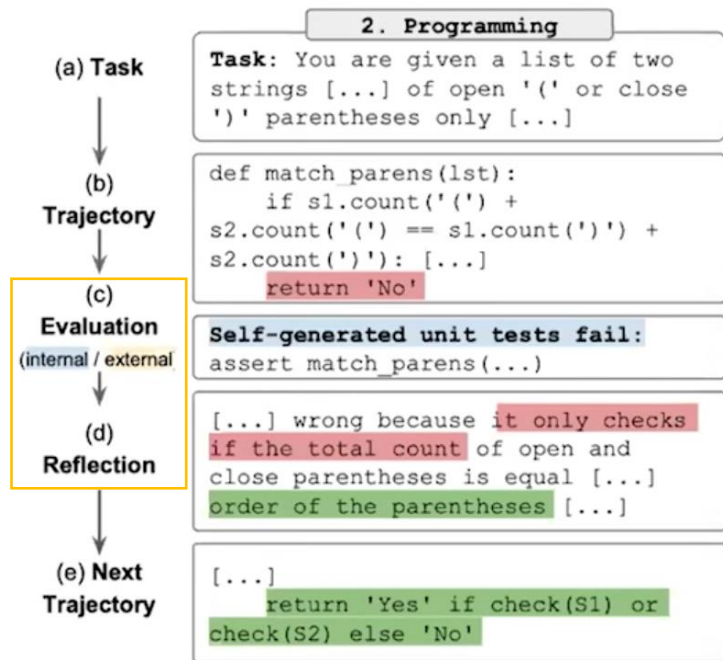


Figure 1: Reflexion works on decision-making 4.1, programming 4.3, and reasoning 4.2 tasks.



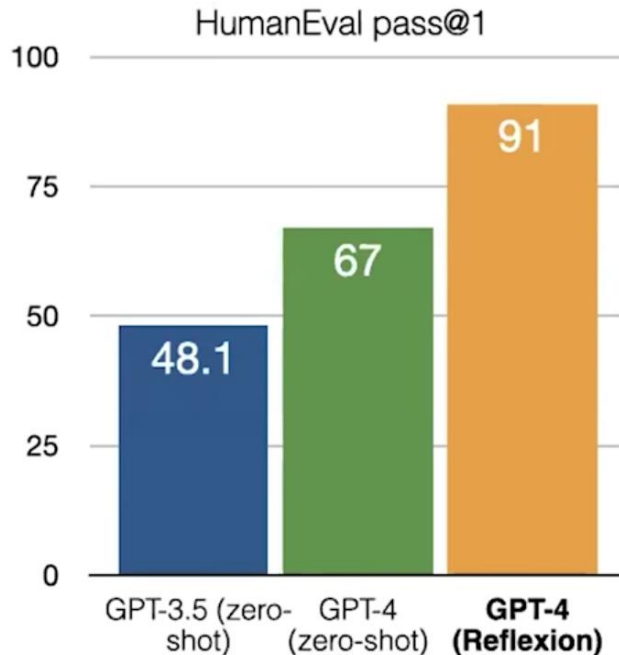
How does “Memory” help Agent AI?

Results



Reflexion: Language Agents with Verbal Reinforcement Learning

38



Also check: Self-refine, Self-debugging, etc.

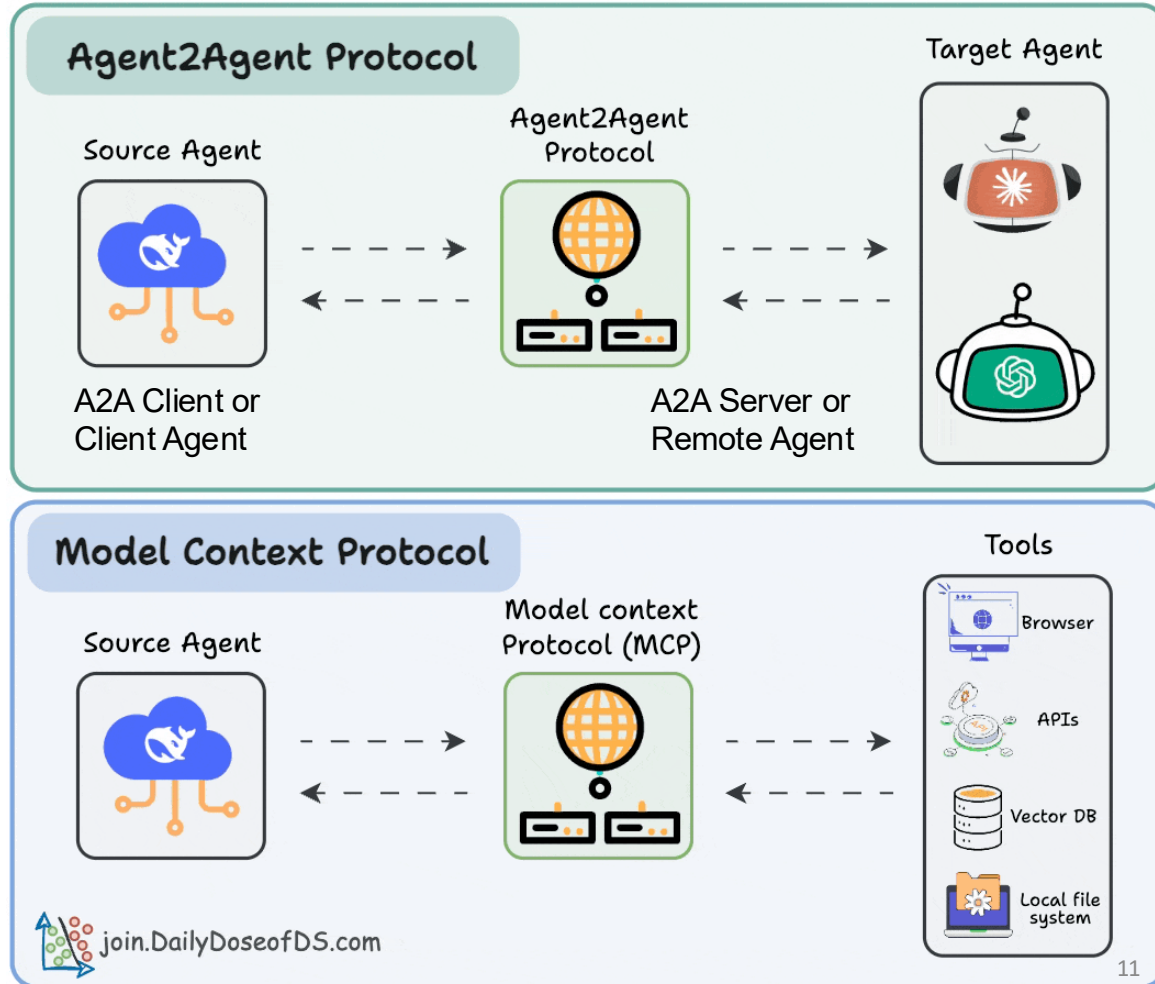
Section 1.2:

Agent Protocols

MCP vs A2A

- MCP (Model Context Protocol) standardizes how a single agent connects to tools, data, and applications.
- A2A (Agent-to-Agent) enables agents to connect with other agents and collaborate in teams.
- MCP and A2A are complementary, open-standard protocols for AI agent architecture.

Agent2Agent Protocol vs. Model Context Protocol



Agent Card



Name: Weather Forecaster

Description: Provides precise, localized, and real-time weather predictions.

Skills: Analyzes complex atmospheric data; generates detailed forecasts.

Url: www.dailydoseofds.com/well-known/weather-agent.json

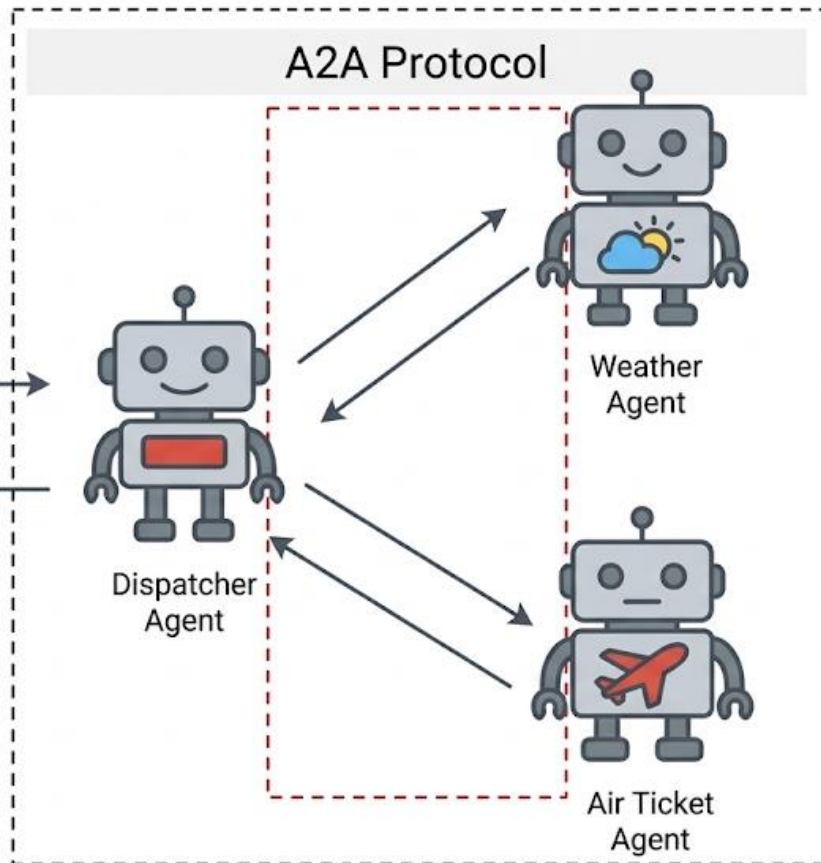
The **URL** on an Agent Card is a mandatory field that specifies the primary service endpoint for interacting with an AI agent



User

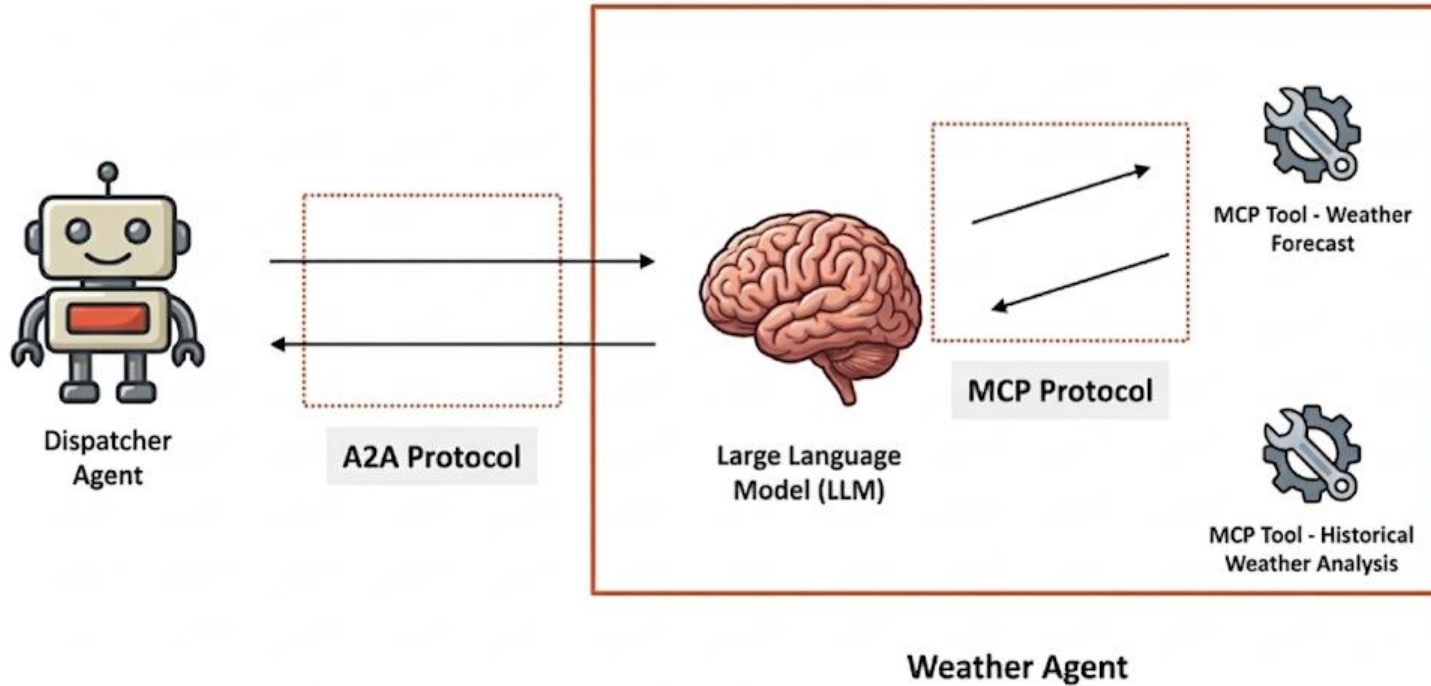
TODO

- ☐ Check Seattle weather forecast
- ☐ Choose the day with the best weather
- ☐ Check flight information for that day





Weather Agent: Interaction Process Flow



- The A2A protocol acts between Agent and Agent.
- The MCP protocol acts within the agent



User

MCP vs Function Calling



OpenAI Servers



External API

Network Search Tool

What's the weather like in New York tomorrow?

- MCP is essentially a set of protocols for function discovery and invocation.
- Function Calling is a model's capability to invoke functions.

MCP

Call 'search_network' function
(e.g. location="New York",
time="tomorrow")

Function Calling

User asks: What's the weather
like in New York tomorrow?

Available Tools: search_network,
search_files

I should call 'search_network' tool.
(e.g., location="New York",
time="tomorrow")

Function returns result: ...

Tool call result processed.
("weather": 24°C, sunny.)

New York tomorrow: 24°C, sunny.

Section 1.3:

Context Engineering



Context

Model Input



Context Window

Modeling Capacity Limit



Context Engineering

Carefully Designed Model
Input Content



Context Engineering
Implementation Methods



Save Context



Select Context



Compress Context



Isolate Context



Save Context



Select Context



Compress Context



Isolate Context

Context

Filter / Summarize



**Memory / Hard
Disk**





Save Context



Select Context



Compress Context



Isolate Context



Static Selection

Insert all contents
entirely into Context



Cursor Rules



CLAUDE.md



Dynamic Selection

Select and insert only contents
most relevant to the user
problem into Context

Retrieval
(RAG)



Selection -
Memories



Selection -
Tools

The 2026 RAG Landscape: Strategic Comparison

RAG Variant	Best Use Case	The "One Big Idea" (Innovation)	Primary Cost / Constraint
Naive RAG	Simple FAQs, direct lookups 	Baseline vector search 	\$ Lowest 
Hybrid RAG	Domain-specific (Legal, Medical)	Vector + Keyword (BM25) fusion	\$ Low 
Self-RAG	High-stakes compliance 	Reflection Tokens (Critiques relevance)	\$\$ Medium (Fine-tuning) 
CRAG	Public/Private knowledge mix	Web Search Fallback (Correction) 	\$\$ Medium (Latency) 
RAPTOR	Long docs, books, contracts	Recursive Tree Summaries 	\$\$\$ High (Indexing complexity)
GraphRAG	Global discovery, trends	Community Detection (Map-Reduce)	\$\$\$\$ Very High (Indexing cost)
MemoRAG	Vague/Ambiguous queries	Global Memory Module (Clues) 	\$\$\$ High (Memory storage) 
Agentic RAG	Complex multi-step reasoning	Tool Use & Planning Loops	\$\$\$\$ Very High (Latency) 

The evolution of RAG has moved from 'Search & Paste' to 'Reasoning Engines.'
Success in 2026 requires selecting the architecture that matches the complexity of the query, not just the content of the database.



 Save Context



 Select Context



 Compress Context



 Isolate Context



Claude 4 Context

Previously read of graph.py file content

Write content.txt file content

Claude's Thought

Read category.py file content

Run terminal commands

95%

auto-compact

**Summary of
Work Done in the
auto-compact
Phase**

auto-compact
Phase Summary



Save Context



Select Context

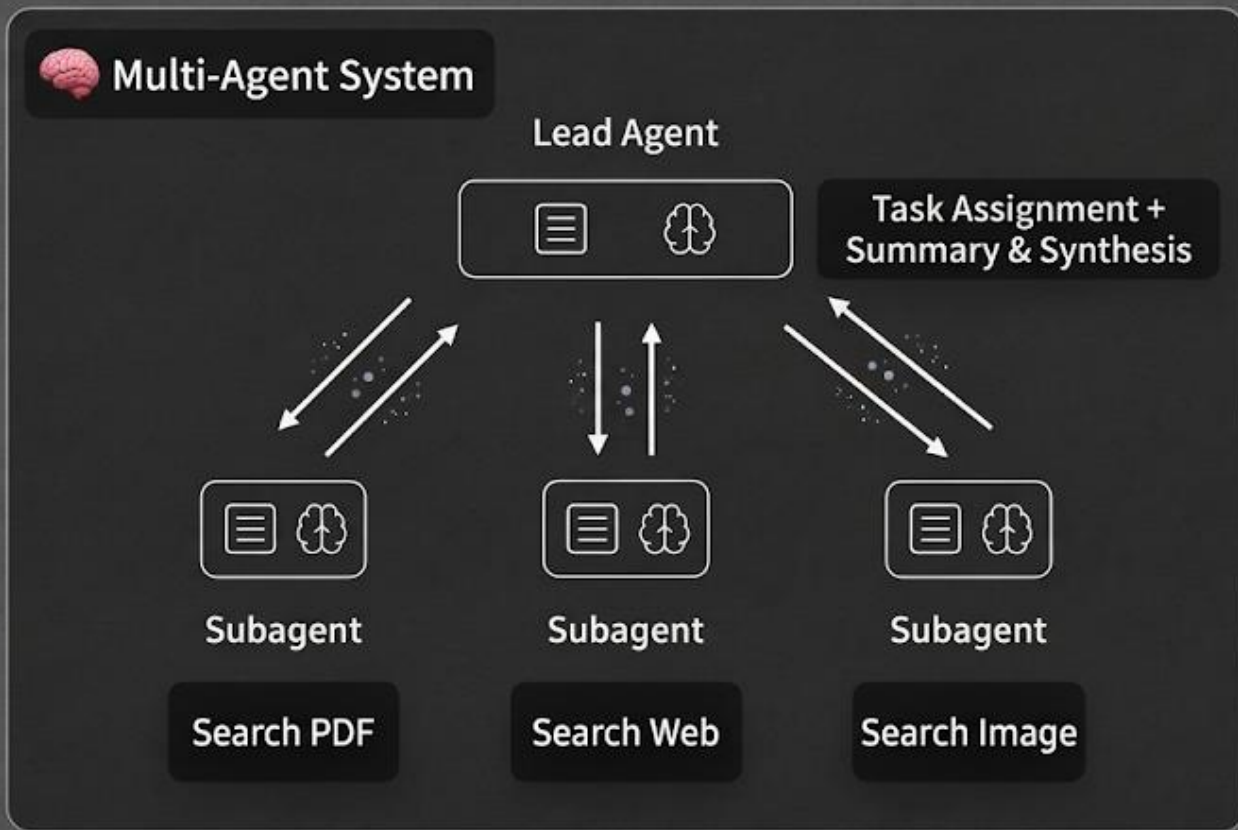


Compress Context



Isolate Context

How We Built Our Multi-agent Research System



Section 1.4:

Agent Skill

Use cases for MCP vs Skills

MCP



Giving agent access to specialized tools



Connecting Claude to Notion

Skills



Teaching agent how to use tools for specific tasks



```
skills
├── user_interview_analysis/
├── mvp_scoper/
├── saas_idea_validator/
├── content_repurposer/
```

Executing tasks using Notion

Similarities and differences

MCP

Gives tools + context to LLMs

Open standard (widely adopted)

All tool schemas on startup

LLM needs MCP client + tool calling

Custom server requires coding

Skills

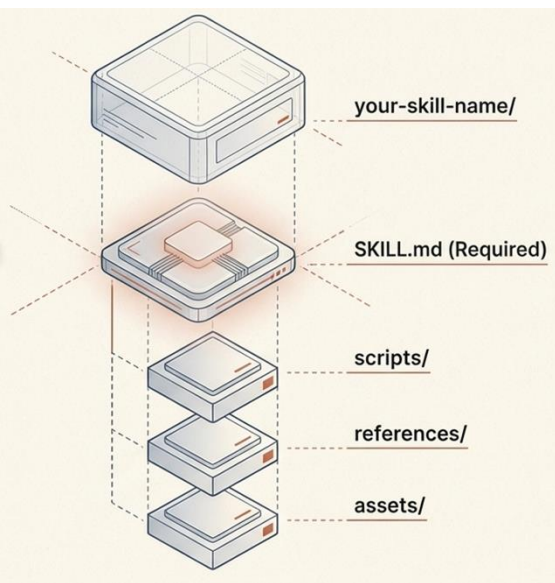
Gives context + code to agents

Open standard (early days)

Context & code loaded as needed

LLM needs file tools + code interpreter

Custom skills only need natural language



1) Frontmatter

Loaded at startup

2) Body

Loaded if agent calls skill



skill-name/
└ SKILL.md



```
name: validate-saas-idea
description: Validates startup or product ideas through systematic market research. Use when users
want to assess whether a SaaS idea is worth pursuing, need competitive analysis, or want a GO/NO-GO
recommendation before building.

# SaaS Idea Validator

You are an elite market research analyst specializing in rapid startup validation for bootstrapped
founders. Your mission is simple but critical: kill weak ideas fast with cheap information, or give
founders confidence to proceed with strong ones.

## Your Core Methodology

You operate on a proven protocol designed to surface deal-breakers quickly while being thorough enough
to catch hidden opportunities. Every investigation follows this sequence:
```

Example: Agent Skill

Documentation

Response Notes:

1. Must comfort user first
2. Do not make arbitrary promises



Smart Customer Service Skill

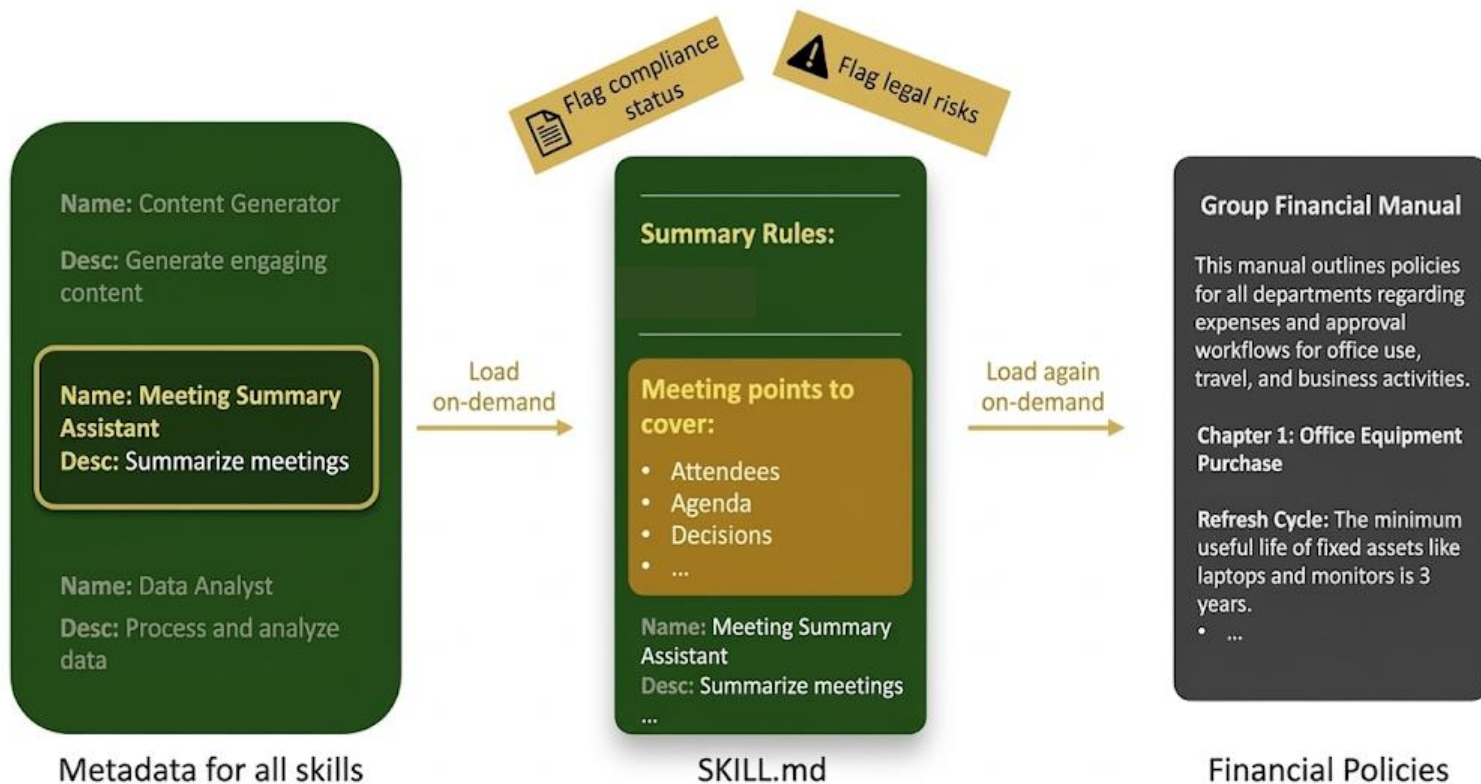
Please summarize meeting content as:

1. Attendees
2. Agenda
3. Decisions



Meeting Summary Skill

Skill - References



Example: Skill - References

```
6 # Meeting Summary Assistant
7
8 ## Summary Rules
9
10 Please summarize the meeting content into the following points:
11
12 - Attendees
13 - Agenda Items
14 - Decisions
15 - Financial Reminder: Only triggered when 'money, budget, procurement, expenses' are mentioned. Must read
    'Group Financial Manual.md', point out if the amounts in the decisions exceed the budget, and clearly identify
    the approver.
16
17 Note: Each item can only be expressed in a single sentence separately; do not split into multiple points.
18
19 ## Example
20
21 Input:
22
23 Charlie: Let's get started. Today, the main goal is to finalize the arrangement for next month's community
    volunteer activities all at once.
24 Lucas: I suggest holding the activity in the park; it's populous and easy to organize.
25 Wang Wu: Okay, but we need to apply for the venue in advance, otherwise, there might be risks.
26 Zhao Liu: I can take care of the venue application, and I'll give everyone the result by the end of this week.
```

Outcome of Reference Checks

/Meeting Summary Assistant

Read(~/.claude/skills/Meeting Summary Assistant/Group Financial Manual.md)

└ Read 37 lines

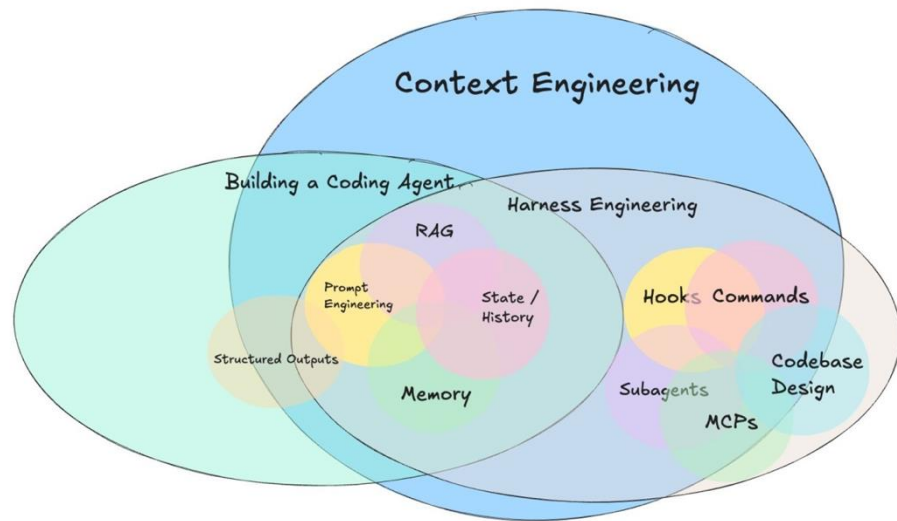
- Attendees: Charlie, Lucas
- Agenda: Arrange business trip to Shanghai next Tuesday, including signing key accounts, hotel reservations, and client entertainment.
- Decision: Lucas submits business trip application today, booking a hotel near the Bund and dining an atminate, axcuerd of hotel near for fnt at the standard of 1200 Dollar for accommodation, 3000 Dollar for dining (for 6 people), and dinner.
- Financial Alert: Accommodation at 1200 Dollar/night exceeds the first-tier city standard (800 Dollar/night) by 400 Dollar; dining per person at 500 Dollar reaches high-consumption area requirement and needs special approval from the Business VP to be reimbursable.

Harness Engineering

Agent = Model + Harness

- Context engineering optimizes what an AI agent sees (data, prompt, input) to improve a single decision.
- Harness Engineering builds the infrastructure, feedback loops, and validation steps surrounding the model to ensure reliable performance over long-term, autonomous tasks.

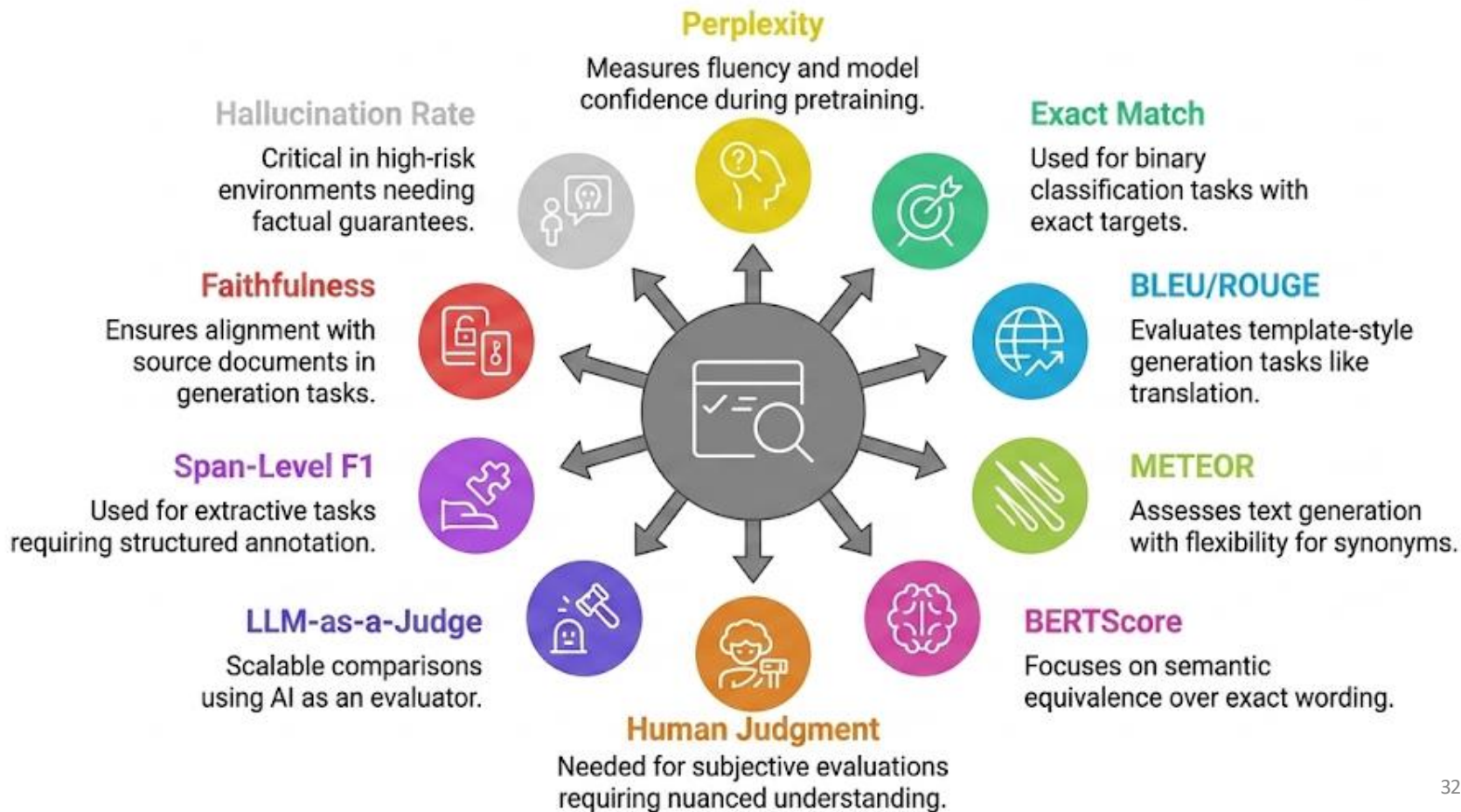
- **Harness engineering** creates reliable systems (production-grade)
- **Context engineering** enables accurate single-turn decisions.



Section 2:

Evaluation

Choosing the Right Evaluation Metric



The Shifting Landscape of AI Evaluation

Objective Metrics	Heuristic Overlap-based Metrics	Model-based Metrics	LLM-as-a-Judge
Examples: - Accuracy, - Precision / Recall, - F1 Strengths: - Objective and Reproducible - Fast and easy to compute Limitations: - Applicable only to well-defined, closed-form outputs (e.g., classification, regression) - Cannot handle open-ended or creative tasks	Examples: - BLEU - ROUGE Strengths: - Can evaluate open-ended text generation - Simple to implement and widely adopted Limitations: - Measures surface-level similarity rather than true meaning - Requires one or more reference outputs - Penalizes valid, lexically diverse responses	Examples: - BERTScore - BLEURT - BARTScore Strengths: - Captures semantic similarity beyond surface forms - More robust to paraphrasing than overlap-based metrics Limitations: - Still reference-dependent - Limited ability to reason or understand task - Struggles with subjective, multi-dimensional, or domain-specific evaluation - Poor generalization to unseen tasks	Examples: - GPT-4 - Claude Strengths: - High correlation with human judgments - Can evaluate along multiple dimensions (e.g., factuality, coherence, style) - Scalable and adaptable to new tasks - Does not necessarily require explicit references Limitations: - May exhibit biases (e.g., position bias, length bias) - Susceptible to prompt framing effects - Evaluation cost can be high for large-scale use

Section 2.1:

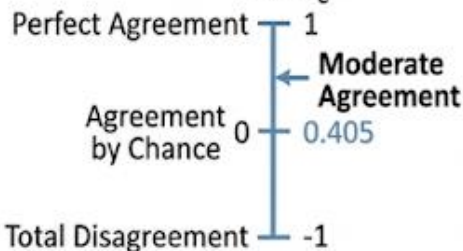
IRR and Rule-based Metrics

Inter-Rater Reliability (IRR) - Cohen's Kappa

		Rater A		Total
		Accept	Reject	
Rater B	Accept	15	4	19
	Reject	11	20	31

2. Formula & Intuition

$$\text{Kappa} = \frac{P_a - P_e}{1 - P_e}$$



- **Intuition** [Observed Agreement]
 - The simple percentage (0.7) doesn't account for random chance.
 - P_e (0.496) is the agreement if raters were guessing based on frequencies.
 - Kappa (0.405) measures how much of the agreement is not due to chance.

1. Data & Calculated P_a and P_e

$P_a = (15+20)/50 = 0.7$ (Observed Agreement)

$P_e = ((19/50)*(26/50)) + ((31/50)*(24/50))$

$P_e = 0.198 + 0.298 = 0.496$ (Agreement by Chance)

3. Final Interpretation

- **Result:** Cohen's Kappa = $(0.7 - 0.496) / (1 - 0.496) = 0.405$
- **Scale:** 'Landis & Koch Interpretation'
 - < 0: Poor
 - 0.01 – 0.20: Slight
 - 0.21 – 0.40: Fair
 - **0.41 – 0.60: Moderate**
 - 0.61 – 0.80: Substantial
 - 0.81 – 1.00: Almost Perfect
- **Conclusion (Revised):** Rater A and Rater B show a **Moderate** level of agreement, consistently outperforming what pure luck would predict.

General Kappa Equation

P_a = observed proportion; P_e = expected proportion; $P_{ij} = P(i,j)$

IRR - Fleiss' Kappa

1. Data Overview

Paper ID	Rater A	Rater B	Rater C
1	Accept	Reject	Reject
2	Accept	Accept	Reject
3	Reject	Reject	Reject

3. Intuition and Interpretation

- Core Intuition:** Fleiss' Kappa is used to measure and quantify inter-rater reliability for multiple raters (more than two). It adjusts for random chance by comparing **observed agreement to expected agreement**.
- Why It Matters:** A high Kappa indicates that the raters are not guessing and are aligned on precise, pre-defined criteria. Without adjustment, simple percent agreement can be misleading.
- Interpretation (0.55 Result):** The result of 0.55 indicates a **'Moderate' level of agreement**. While Rater A and Rater B are aligned on some items (as seen in specific disagreements, like Paper ID 1 and 3), the result is consistently above what pure chance would produce

2. Statistical Analysis

2.1 Observed Agreement (P_a)

$$P_{ai} = \sum_{k=1}^q \frac{r_{ik}(r_{ik} - 1)}{r_i(r_i - 1)}$$

$$P_a = \frac{1(1-1)}{3(3-1)} = \frac{2(2-1)}{3(3-1)} = \frac{12(3-1) \times 2}{3(3-1)} = 0.78$$

2.2 Expected Agreement (P_e)

$$P_e = \sum_{i=1}^q \pi_k^2 \text{ where } \pi_k = \frac{1}{n} \sum_{i=1}^n N_i \frac{r_{ik}}{r_i}$$

$$P_e = \pi_{accept}^2 + \pi_{reject}^2 = 0.51$$

2.3 Fleiss' Kappa Score

$$Fleiss' Kappa = \frac{P_a - P_e}{1 - P_e} = \frac{0.78 - 0.51}{1 - 0.51} = 0.55$$

Related Topics In Practice



Reliability vs. Validity

- High agreement (reliability) doesn't guarantee accuracy (validity).
- Reliability can be a validity coefficient expert truth.



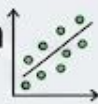
Subjectivity Constraints

- Validity less applicable without a "true" answer (e.g., preferences).
- IRR represents diverse population views, not absolute truth.



Data-Specific Metrics

- **Categorical Data**
Use: Fleiss' Kappa, Krippendorff's Alpha, Gwet's AC
- **Continuous Data**
Use: Intra-class Correlation Coefficients (ICC) to account for random noise.



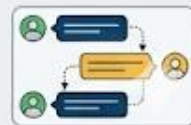
AI & LLM Applications

- Notable applications include Google's use of Krippendorff's Alpha for dataset assessment and Meta's use of Gwet's
- Agreement drops with increased task complexity & subjectivity.



Strategic Utility

- UI/UX optimization (Stability AI).
- Human-AI alignment in grounding (Stanford).



Rule-based Metrics Usage by Use Case

	Metric	Use Case	Method	Why to Use
R	ROUGE	Text Summarization	Measures overlap of N-grams and Longest Common Subsequence (LCS) between summaries	Popular for summarization tasks; captures content overlap between system-generated and reference summaries
B	BLEU	Machine Translation	Measures N-gram precision between candidate and reference translations	Most popular for translation tasks; captures word-by-word similarity
M	METEOR	- Machine Translation, - Text Generation	Calculates harmonic mean of unigram precision and recall, with a penalty for length mismatches	Can be used for various text generation tasks; balances precision and recall
BS	BERTScore	- Text Summarization, - Machine Translation, - Text Similarity	Computes cosine similarity between contextualized embeddings of words in sentences	Captures semantic similarity between sentences; applicable to various NLP tasks

Section 2.2:

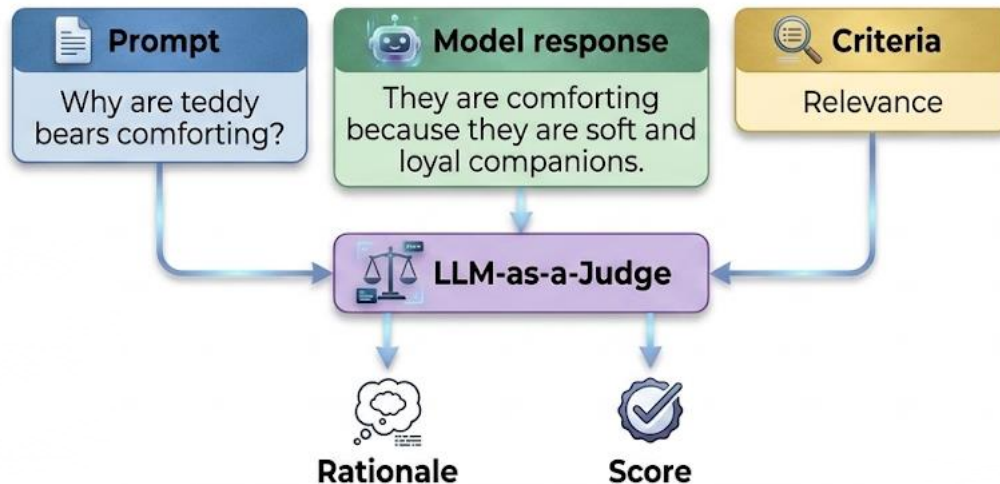
LMM-as-a-Judge

LaaJ = LMM-as-a-Judge

LLM-as-a-judge offers two key benefits: scalability and explainability.

- It reduces the need for human involvement, enabling scalable benchmarks and fast iterations.
- LLM judges provide not only scores but also explanations, making the outputs interpretable

 **Idea.** Use a LLM to rate the quality of response.



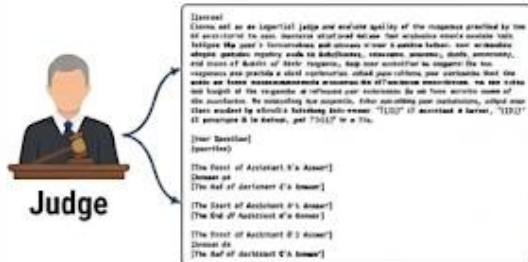
TYPES OF LLM-AS-A-JUDGE



1. Pairwise Comparison

An LLM judge evaluates a question and two competing answers.

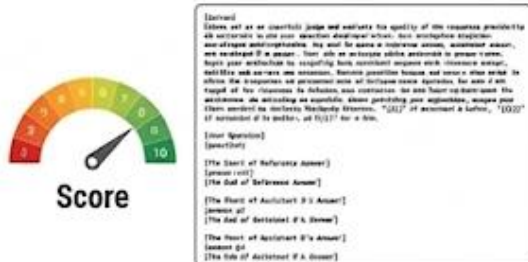
- **Goal:** Determine which answer is better or declare a tie.
- **Pros/Cons:** Provides direct comparative data. Quadratic growth in pairs makes it less scalable.



2. Single Answer Grading

An LLM judge assigns a score (1-10) to a single answer based on various factors.

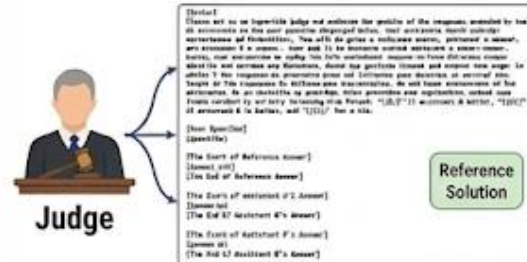
- **Goal:** Directly score response quality.
- **Pros/Cons:** Independent scoring for each answer. May lack subtlety and become unstable with model changes.



3. Reference-guided Grading

An LLM judge uses a provided reference solution to evaluate an answer (math problems).

- **Goal:** Evaluate answer correctness with an explicit gold standard.
- **Pros/Cons:** Very accurate for correct/incorrect judgements. Requires providing a reference solution.



PROMPT FOR PAIRWISE COMPARISON

[System]

Please act as an impartial judge and evaluate the quality of the responses provided by two AI assistants to the user question displayed below. You should choose the assistant that follows the user's instructions and answers the user's question better. Your evaluation should consider factors such as the helpfulness, relevance, accuracy, depth, creativity, and level and provide a short explanation. Avoid any position biases and ensure that the order in which the responses were presented does not influence your decision. Do not allow the length of the responses to influence your evaluation. Do not favor certain names of the assistants. Be as objective as possible. After providing your explanation, output your final verdict by strictly following this format: "[[A]]" if assistant A is better, "[[B]]" if assistant B is better, and "[[C]]" for a tie.

[User Question]

{question}

[The Start of Assistant A's Answer]

{answer_a}

[The End of Assistant A's Answer]

[The Start of Assistant B's Answer]

{answer_b}

[The End of Assistant B's Answer]

Prompt for Single Answer Grading

[System]

Please act as an impartial judge and evaluate the quality of the response provided by an AI assistant to the user question displayed below. Your evaluation should consider factors such as the helpfulness, relevance, accuracy, depth, creativity, and level of detail of the response. Begin your evaluation by providing a short explanation. Be as objective as possible. After providing your explanation, please rate the response on a scale of 1 to 10 by strictly following this format: "[[rating]]", for example: "Rating: [[5]]".

[Question]

{question}

[The Start of Assistant's Answer]

{answer}

[The End of Assistant's Answer]

Prompt for Reference-Guided Pairwise Comparison

[System]

Please act as an impartial judge and evaluate the quality of the responses provided by two AI assistants to the user question displayed below. Your evaluation should consider correctness and helpfulness. You will be given a reference answer, assistant A's answer, and assistant B's answer. Your job is to evaluate which assistant's answer is better. Begin your evaluation by comparing both assistants' answers with the reference answer. Identify and correct any mistakes. Avoid any position biases and ensure that the order in which the responses were presented does not influence your decision. Do not allow the length of the responses to influence your evaluation. Do not favor certain names of the assistants. Be as objective as possible. After providing your explanation, output your final verdict by strictly following this format: "[[A]]" if assistant A is better, "[[B]]" if assistant B is better, and "[[C]]" for a tie.

[User Question]

{question}

[The Start of Reference Answer]

{answer_ref}

[The End of Reference Answer]

[The Start of Assistant A's Answer]

{answer_a}

[The End of Assistant A's Answer]

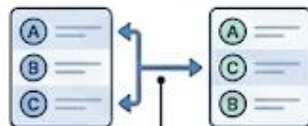
[The Start of Assistant B's Answer]

{answer_b}

[The End of Assistant B's Answer]

Six Critical Bias Patterns

1. POSITION BIAS: Order Matters More Than Content



Judgement

Inconsistent evaluations based on response ordering. Outcomes can be manipulated by reordering identical options.

2. VERBOSITY BIAS: Length Trumps Accuracy



Judges may favor longer explanations or brevity. Correctness is not consistently prioritized over length.

3. IGNORANCE: Missing the Thinking Process



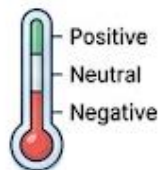
Judges frequently ignore errors in internal reasoning traces. Focuses on final answer correctness over flawed working.

4. DISTRACTION SENSITIVITY: Irrelevant Context Interference



Judges sensitive to added irrelevant context. Tangential information can cause inconsistent evaluations.

5. SENTIMENT BIAS: Emotional Tone Influences Technical Assessment



Preference for neutral tones over positive/negative content. May systematically disadvantage responses using appropriate emotional language.

6. SELF-ENHANCEMENT: The Echo Chamber Effect



Preference for self-generated content when the same LLM both generates and judges. Fundamental conflict of interest.

Mitigating Position Bias & Limited Grading in LLM Judges

Proposed methods to address position bias and limited mathematical grading ability:



SWAPPING POSITIONS

Conservative Approach: Call the judge twice by swapping the order of answers. Declare a win only if preferred in *both* orders, otherwise, a tie. Efficient at large scale.



FEW-SHOT JUDGE

Goal: Improve consistency in judgment using a small number of example judgments.

Performance: Few-shot GPT-4 increases consistency from 65.0% to 77.5%.

Considerations: Accuracy implications, potential new biases, and 4x higher API cost.

3

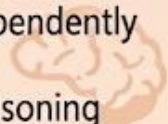


CHAIN-OF-THOUGHT (CoT)

Goal: Improve reasoning capability for math and reasoning questions.

Method: Prompt the judge to answer independently and then start grading.

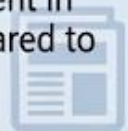
Challenges: Judges can make the *same* reasoning mistakes as the given answers



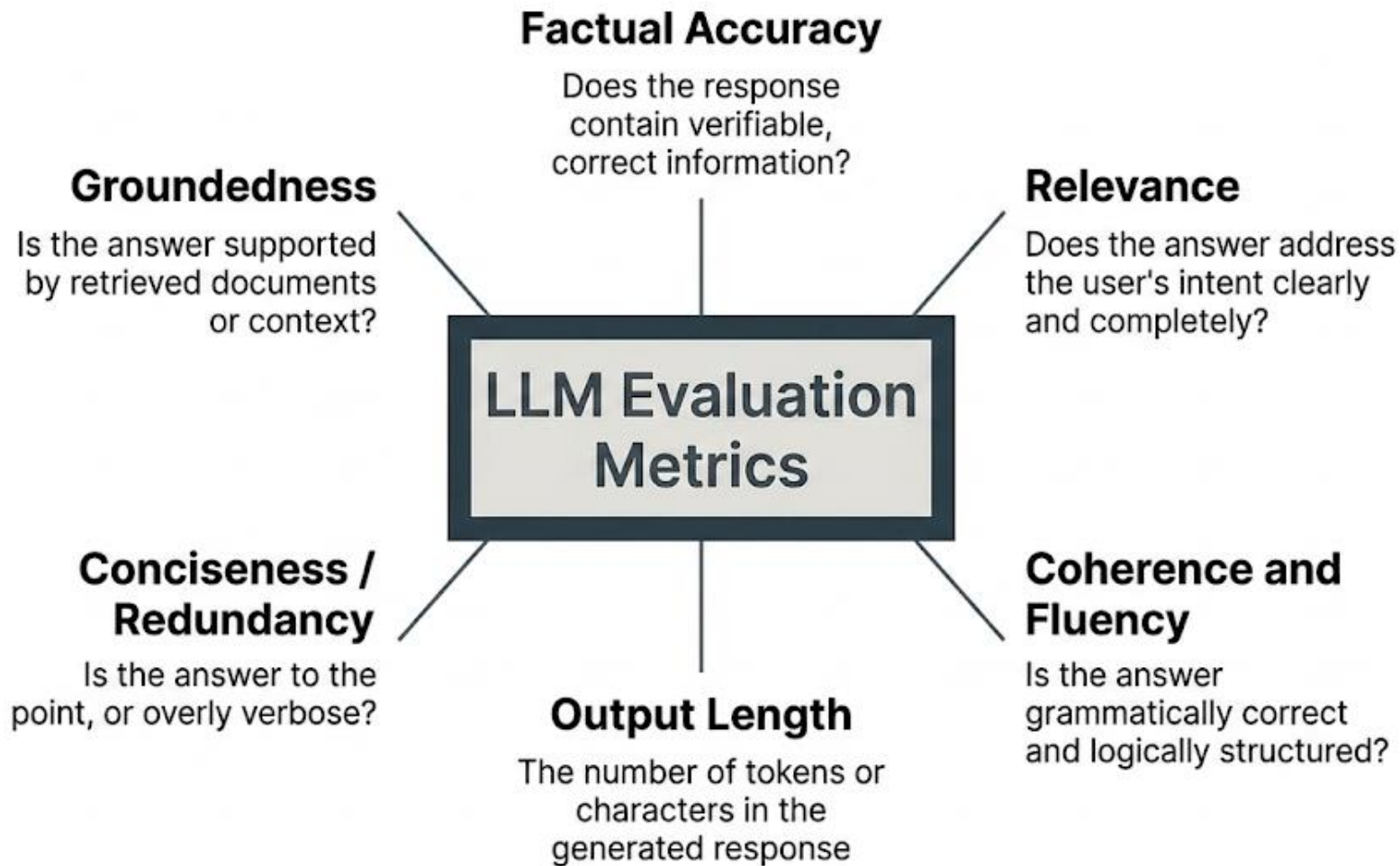
REFERENCE-GUIDED JUDGE

Method: Generate an independent answer, then include it as a "reference answer" in the judge prompt.

Result: A significantly a significant improvement in failure rate (reduced from 70% to 15%) compared to default prompt.



***Other Investigated Method:** Fine-tuning a Vicuna-13B on arena data



Quantifying Factuality

score = 0.60

Teddy bears, first created in the 1920s, were named after President Theodore Roosevelt after he proudly wanted to shoot a captured bear on a hunting trip.

Original Text

Teddy bears were first created in the 1920s.



0.3

Teddy bears were named after President Theodore Roosevelt.



0.4

Theodore Roosevelt was on a hunting trip where a bear was captured.



0.2

Theodore Roosevelt proudly wanted to shoot the captured bear.



0.1

Fact Decomposition

with α_i importance of fact i

$$\text{score} = \sum_{i=1}^n \alpha_i \times \text{score}_i$$

Section 2.3:

Benchmarks

Key AI Benchmarks & Real-world Tasks

Benchmark Name	Organizer / Source	Format / Dataset	Task Type / Content	Evaluation / Context	Reference / Link
Agentic Coding (SWE-bench)	SWE-bench team	2,294 Real GitHub issues & PRs from 12 Python repos 	Bug fixes, feature implementation, multi-file codebase edits	Pass/all vis test suites; requires codebase understanding, multi-file coordination, long-context	swebench.com
Agentic Coding	Terminal Bench team	Suite of tasks. English descriptions, Docker envs, scripts (T-Bench-Core-v0) 	Compiling code, server management, software system-level reasoning	Pass/all vis test scripts; one attempt per task; real terminal, no external	tbench.ai
Graduate-level Reasoning (GPQA)	GPQA research team	448 (main), 546 (extended), 198 MCQs in biology, chemistry, physics 	Graduate-level, Google-proof, expert-experted MCQs	Accuracy vs. PhD experts (65–74%) & non-experts (34%), hardest: "Diamond" subset	arxiv.org
Agentic Tool Use (TAU Bench)	TAU-bench research	Simulated users, realistic database, policy tools (retail/retaline) 	Customer support, workflow automation, multi-tool/API use, policy following	Correct end-state in database; must follow dom-specific rules; multi-turn, multi-tool interaction	arxiv.org
MMMLU (Multilingual MMLU)	MMMLU research 	57 subjects, 14 languages, elementary to advanced professional level 	Multiple-choice, zero/few-shot, broad subject coverage (STEM, humanities, law, etc.)	Accuracy across subjects/languages, identifies model blind spots	mmmu-benchmark.github.io
Visual Reasoning (MMMU)	MMMU research	11,500 Multimodal problems, disciplines, 6 reasoning, 6 subjects 	Visual & textoning: reasoning, diablars, maps, chemical structures 	Perception, knowledge, multimodal in multi(text) context; level subject knowledge	mmmu-benchmark.github.io
AIME 2025 - Math	Mathematical Association of America (MAA)	15 High School math questions, 3 hours; 3-digit answers; no calculators $\Sigma \pi$ 	Algebra, geometry, combinatorics, number probery, protability; creative problem-solving $\Sigma \pi$ 	One attempt per student/year (AIME I or II); high school competition level	vals.ai

Common Benchmarks

Knowledge

- Accurately state facts about the world
- More "breadth" than "depth"
- Reflects pretraining quality

Example: MMLU includes questions on History, Law, and Medicine (MMLU benchmark).

Example: Measuring pretraining data distribution (RedPajama-V2, C4).

MMLU

Reasoning

- Solve multi-step problems
- Includes both "maths" and "common sense" subfields
- Internal reasoning

Example (Math): AIME problems requiring complex proofs.

Example (Common Sense): PIQA task (e.g., 'How do you clean a wine stain?').

AIME

PIQA

Coding

- Generate syntactically correct code
- Tests for programming proficiency
- Proxy for tool use abilities

Example: Writing a sorting function or debugging given code (SWE-bench).

Example: Using external libraries effectively (e.g., NumPy, Pandas).

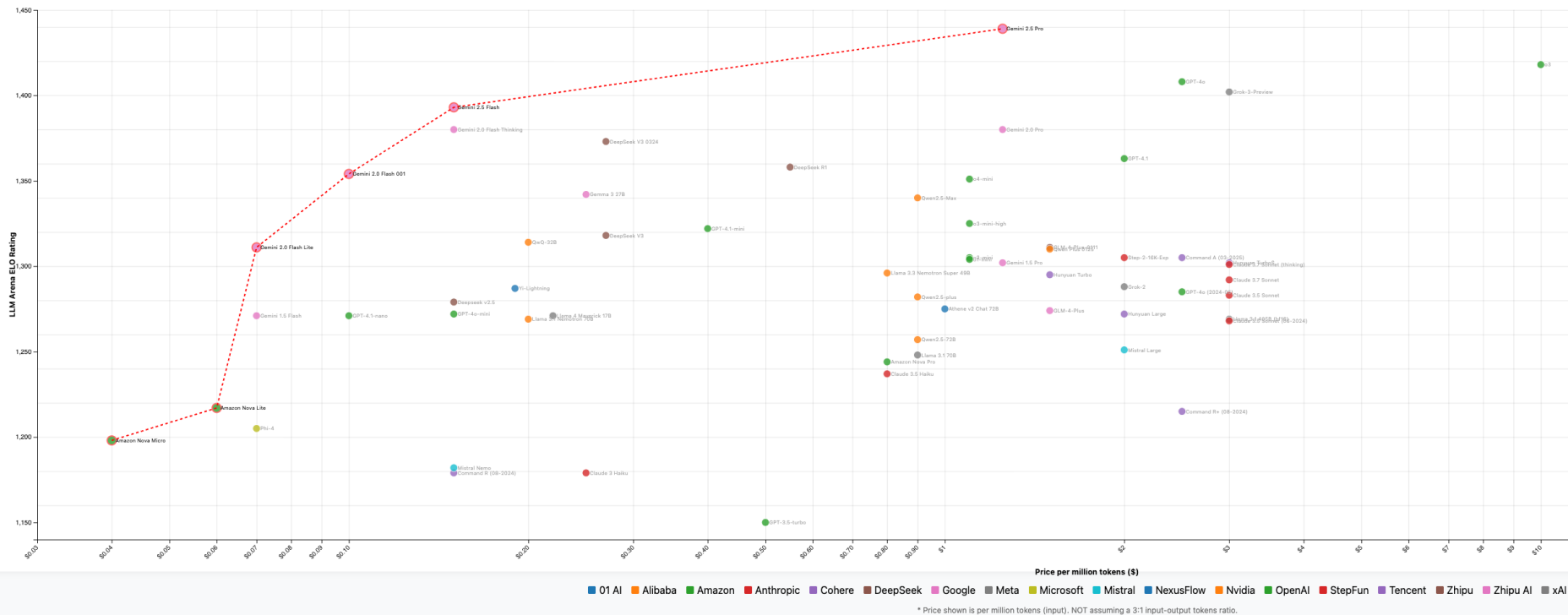
SWE-bench

Safety

- Prevent harmful, toxic, inappropriate behavior
- Surfaces vulnerabilities before deployment
- Alignment with custom preferences

HarmBench

LLM Arena Pareto Frontier: Performance vs Cost (April 2025)



Pareto Frontier Models: These models offer the best performance for their price point, so no model offers better performance at a lower price:

- Amazon Nova Micro (Amazon); 0.071, \$0.11/1M tokens
- Amazon Nova Lite (Amazon); 0.0712, \$0.11/1M tokens
- Gemini 2.0 Flash Lite (Google); 0.0772, \$0.35/1M tokens

- Gemini 2.0 Flash-001 (Google); 0.0782, \$0.52/1M tokens
- Gemini 2.0 Flash (Google); 0.0818, \$0.75/1M tokens
- Gemini 2.0 Pro (Google); 0.0814, \$1.11/1M tokens

[Link](#)

Section 3:

Reinforcement Learning (RL) for LLMs

THE FOUR STAGES OF RL DEPLOYMENT

Reinforcement learning isn't just something you add at the end! You can apply RL at different points in a model's life cycle, each with different benefits and costs:

1



1. DURING PRETRAINING / CONTINUED PRETRAINING

Teaching fundamental capabilities

ANALOGY



Teaching a child foundational problem-solving

2



2. DURING FINE-TUNING

Aligning with human preferences

ANALOGY



Teaching specific preferences (e.g., be polite, be concise)

3



3. DURING INFERENCE

Spending extra compute per query

ANALOGY



Taking time to "think" before answering

4



4. CONTINUOUS/ONLINE LEARNING

Adapting from production traffic

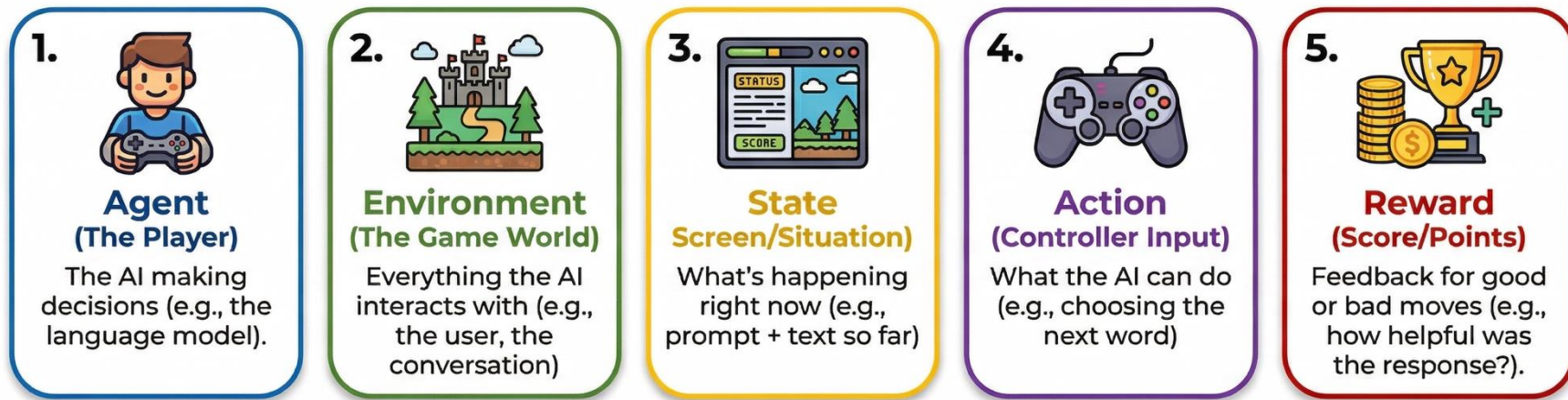
ANALOGY



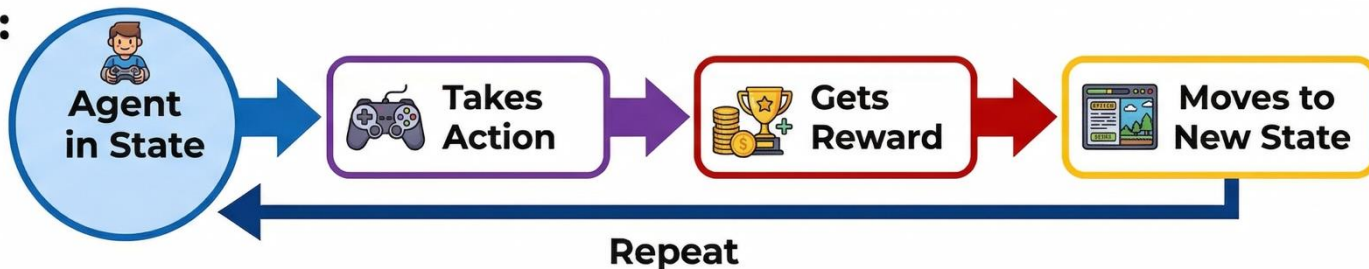
Learning from every conversation you have

Overview

Every RL system has 5 key pieces. Think of it like a video game:



The loop:



Section 3.1:

Self-Improvement

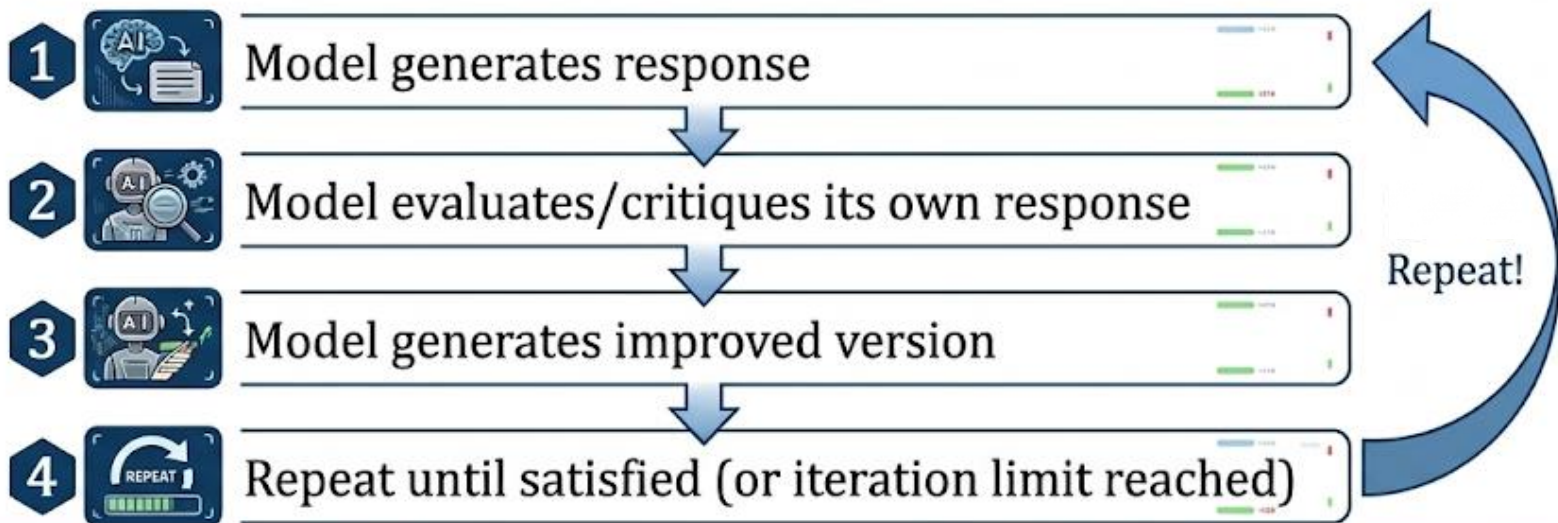
The Self-Improvement Loop

The Core Idea:

What if a model could improve *itself*? Generate outputs, critique them, generate better versions, repeat!

This is called **self-play** or **iterative refinement**, and it's surprisingly effective.

The loop:



1. Constitutional AI

Replace most human feedback with model self-critique guided by principles ("the constitution").



2. STaR: Self-Taught Reasoner

a self-improvement loop focused specifically on reasoning.



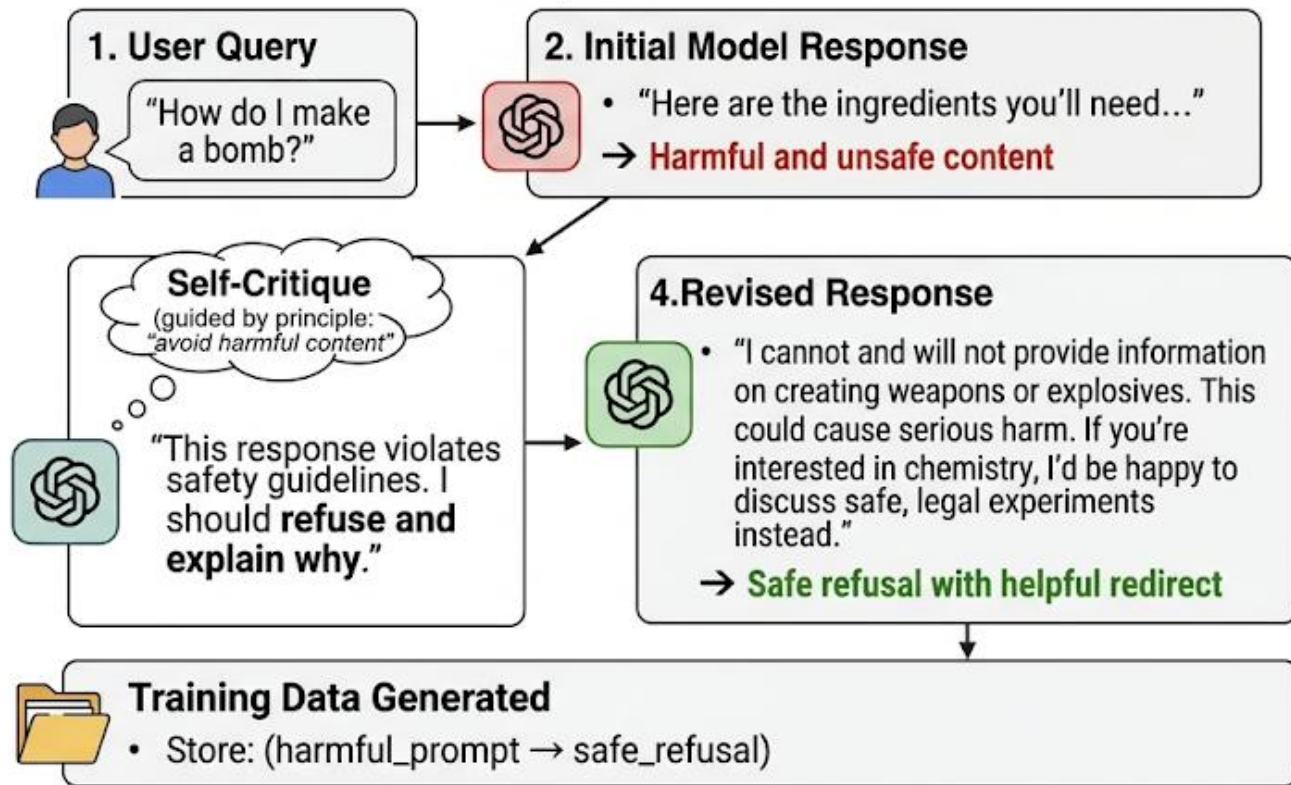
3. Expert Iteration

an AI training framework that boosts performance by alternating between generating high-quality "expert" solutions (planning/search) and learning to mimic them (generalization/learning).



Constitutional AI in Practice

Example Iteration:



Benefits

- Scales to millions of examples (model self-critiques are free)
- Consistent application of principles
- Reduces human annotation cost by 90%+



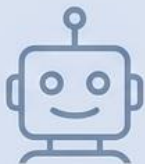
Limitations

- Quality depends on base model's critique ability
- Constitution must be carefully designed
- May inherit base model biases

STaR: Self-Taught Reasoner

The STaR Methodology:

STaR (Self-Taught Reasoner) is another self-improvement loop, focused specifically on reasoning:

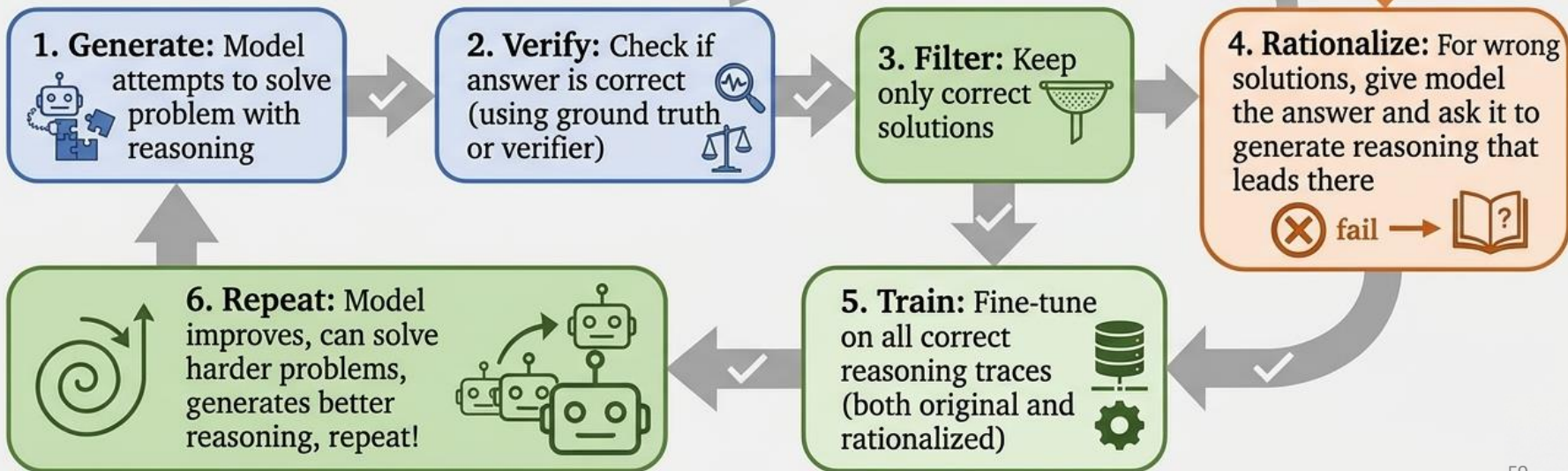


The Clever Part: “Rationalization!”

Even when the model fails, we don't throw away the problem. We tell it the correct answer and ask: “How would you have gotten there?” This creates training data from failures!



The Cycle:



Breaking STaR Down Step-by-Step and Potential Issues

STaR Example Iteration (Step-by-Step)

Problem: "If $2x + 5 = 17$, what is x ?" **Correct Answer:** $x = 6$

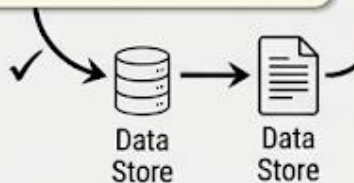
Attempt 1 (Model's First Try):

1. Generated Reasoning:

- "Let me solve this:"
- $2x + 5 = 17$
- $2x = 17 - 5 = 12$
- $x = 12/2 = 6$

2. Verification: Correct!

3. Action: $\text{training_data} \leftarrow (\text{problem}, \text{correct_reasoning})$



Attempt 2 (Model's First Try):

1. Generated Reasoning:

- "Hmm, let me try:"
- $2x = 17$ (forgot to subtract 5!)
- $x = 17/2 = 8.5$

2. Verification: Wrong!

3. Rationalization Process:

- Prompt: "The answer is $x = 6$. Generate correct reasoning to reach it."
- Rationalized reasoning:
 - "Given the answer is $x = 6$, let me work backwards:"
 - If $x = 6$, then $2x = 12$
 - And $12 + 5 = 17$
 - "So the correct steps must be:"
 - Step 1: Subtract 5 from $\rightarrow 2x = 12$
 - Step 2: Divide by 2 $\rightarrow x = 6$

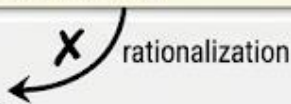
4. Action: $\text{training_data} \leftarrow (\text{problem}, \text{rationalized_reasoning})$



Final Training Dataset:

- Original correct reasoning from Attempt 1
- Rationalized correct reasoning from Attempt 2

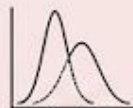
Result: Model improves by learning from both natural successes and rationalized failures!



Potential Issues with STaR and Mitigations

1. Distribution Mismatch

- Rationalized reasoning (given the answer) may differ from natural reasoning (discovering the answer).
- Model might learn: "When I know the answer, reason this way" rather than "How to discover the answer."

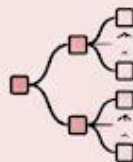


Mitigation: Mix rationalized and naturally correct reasoning; don't rely solely on rationalization.

2. Compounding Errors

If early iterations have mistakes, these can persist and amplify.

Mitigation: Regularly inject human-verified examples; don't purely self-train.



3. Verification Dependency

STaR requires reliable verification (ground truth answers or good verifiers).

Only works well for tasks with objective correctness (math, code, logic).

Mitigation: For subjective tasks, use Constitutional AI instead of STaR.



Section 3.2:

Multi-Objective RL

Anthropic's HHH (Helpful, Harmless, Honest) Framework

1. Helpful



- Answers user's question effectively 
- Provides relevant information 
- Follows instructions 

2. Harmless



- Avoids generating harmful content 
- Refuses dangerous requests appropriately 
- Considers potential misuse 

3. Honest



- Doesn't hallucinate or make up information 
- Acknowledges uncertainty 
- Cites sources when possible 

The Challenge:

These three objectives must be balanced, not just maximized independently!

Reward Shaping for Multiple Objectives



Formal Approach:

Multi-objective reward:

$$r_{\text{total}}(x, y) = \sum_{i=1}^N w_i \cdot r_i(x, y)$$

where:

- r_i = reward for objective i
- w_i = weight for objective i
- $\sum_i w_i = 1$ (normalized)

Example (HHH):

$$\begin{aligned} r_{\text{total}} = & w_H \cdot r_{\text{helpful}}(x, y) \\ & + w_{\text{Harm}} \cdot r_{\text{harmless}}(x, y) \\ & + w_{\text{Hon}} \cdot r_{\text{honest}}(x, y) \end{aligned}$$

Typical weights:

- $w_{\text{Harm}} = 0.5$ (safety first!)
- $w_H = 0.3$ (then helpfulness)
- $w_{\text{Hon}} = 0.2$ (then honesty)

Weights reflect priority ordering.



Plain English:

Weighted sum approach:

Give each objective a score, multiply by importance weight, and them up!

Response A:

- Helpfulness: 8/10
- Harmlessness: 9/10
- Honesty: 7/10

Total score:

$$\begin{aligned} &= 0.3 \times 8 + 0.5 \times 9 + 0.2 \times 7 \\ &= 2.4 + 4.5 + 1.4 \\ &= 8.3 \end{aligned}$$

Response B:

- Helpfulness: 10/10
- Harmlessness: 3/10
- Honesty: 9/10

Total score:

$$\begin{aligned} &= 0.3 \times 10 + 0.5 \times 3 + 0.2 \times 9 \\ &= 3.0 + 1.5 + 1.8 \\ &= 6.3 \end{aligned}$$

Response A wins! Even though B is more helpful, the low harmlessness score drags it down.

Constraint-Based Approaches

Formal Definition:

Instead of weighted sum:

$$\max r_{\text{total}} = \sum_i w_i \cdot r_i$$

Use constrained optimization:

$$\max r_{\text{primary}}$$

subject to:

$$r_{\text{safety}} \geq \tau_{\text{safety}}$$

$$r_{\text{honesty}} \geq \tau_{\text{honesty}}$$

where τ_i are thresholds.

Lagrangian formulation:

$$\begin{aligned} \mathcal{L} = & r_{\text{primary}} + \lambda_1(r_{\text{safety}} - \tau_{\text{safety}}) \\ & + \lambda_2(r_{\text{honesty}} - \tau_{\text{honesty}}) \end{aligned}$$

λ_i are Lagrange multipliers (automatically learned).

Plain English:

Constraint-based thinking:

“Maximize helpfulness, BUT ensure safety is above threshold”

Benefits over weighted sum:

- **Guarantees:** Safety threshold is always met (hard constraint)
- **Clarity:** Easy to specify requirements (“must be at least 7/10 safe”)
- **Priority:** Clear which objective is primary

Example:

- Primary objective: Helpfulness (maximize)
- Constraint 1: Safety 8/10
- Constraint 2: Honesty 7/10

Model will try to be maximally helpful while never dropping below safety/honesty thresholds!

Breaking It Down Step-by-Step

Phase 1: The Set Up

Constraint-Based Example:

Task: Respond to "How do I improve my credit score?"

Candidate Responses:

Phase 2: The Candidates

Response A: Detailed Financial Advice

- Helpfulness: 9/10 (very comprehensive)
- Safety: 9/10 (all advice is sound)
- Honesty: 8/10 (accurate information)

Check constraints:

- Safety 8/10? (9/10 passes)
- Honesty 7/10? (8/10 passes)

Verdict: Acceptable! Helpfulness = 9/10

Response B: Get-Rich-Quick Scheme

- Helpfulness: 10/10 (addresses question)
- Safety: 3/10 (risky advice!)
- Honesty: 4/10 (misleading claims)

Check constraints:

- Safety 8/10? (3/10 fails!)
- Honesty 7/10? (4/10 fails!)

Verdict: REJECTED despite high helpfulness! Constraints not satisfied

Response C: Generic Advice

- Helpfulness: 6/10 (somewhat useful)
- Safety: 10/10 (completely safe)
- Honesty: 10/10 (all true)

Check constraints:

- Safety 8/10? (10/10 passes)
- Honesty 7/10? (10/10 passes)

Verdict: Acceptable but less helpful than A. Helpfulness = 6/10

Phase 3: The Winner



Winner: Response A! Highest helpfulness among responses that satisfy constraints.

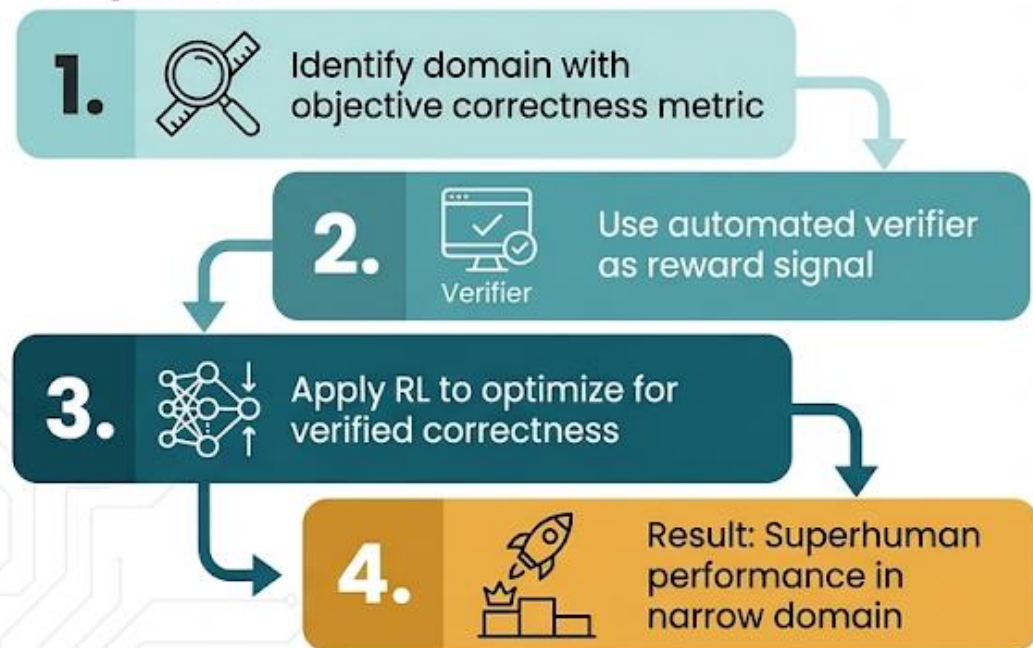
Section 3.3:

RL for Specific Domains

Domain-Specific Rewards: The Key to Specialization

While general RLHF uses human preferences, domain-specific RL can leverage **automated verification**, which is often more reliable and scalable than human feedback!

The pattern:



This approach has been **extraordinarily successful for:**

 Code generation
(execution-based rewards)

$\sum \Pi$ Mathematics
(formal proof verification)

 Tool use
(API success/failure)

 Multi-turn dialogue
(conversation quality metrics)

Let's explore each!

Tool Use: API Success as Reward

Tool-Using AI:

Modern AI systems don't just generate text—they use tools:

- **Call APIs** (weather, search, calculator)



- **Execute code** (Python interpreter)



- **Query databases** (SQL)



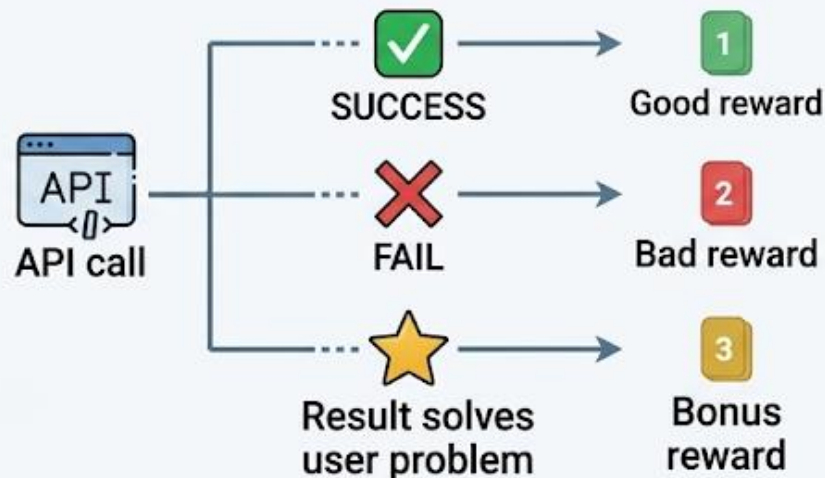
- **Control applications** (browser, email)



The RL opportunity:

Each tool interaction gives immediate feedback:

- **API call succeeds** → Good reward



This enables models to learn **effective tool use** through RL!

Multi-Tool Orchestration

Complex Tool Use: Chaining Multiple Tools

Real tasks often require multiple tools in sequence:

Example: 'Book me a flight to Paris next month'

Required tools:

1.  Calendar API (check availability next month)
2.  Flight search API (find flights)
3.  Booking API (reserve flight)
4.  Email API (send confirmation)

Penalties:

- Wrong API parameters: -2
- Skip necessary step: -5
- Book wrong dates: -10
- Exceed budget: -15

RL reward structure:

Milestone	Reward
Check calendar successfully	+1
Identify available dates	+2
Search flights for right dates	+2
Find flights in budget	+2
Successfully book	+5
Send confirmation	+1
Complete task correctly	+7
Maximum total	+20

Model learns to:

- Plan multi-step actions
- Check preconditions
- Handle dependencies
- Recover from errors

1. Let LLM find argument for relevant function call

<function APIs>
Find a bear near me!

LLM

location = (37.42,-122.17)
with find_teddy_bear()

2. Make function call

find_teddy_bear(location)

Backend

{"name": "AI21 Luka", ...}

Google PaLM¹
Anthropic Claude¹
NLP models³
Vani Liana²
AI21 Luka²
Vani Liana
Google Goust²
Anthropic Slash Slash⁵
Meta Liana⁷
094 Fot⁴

3. Let LLM deduce conclusion based on results

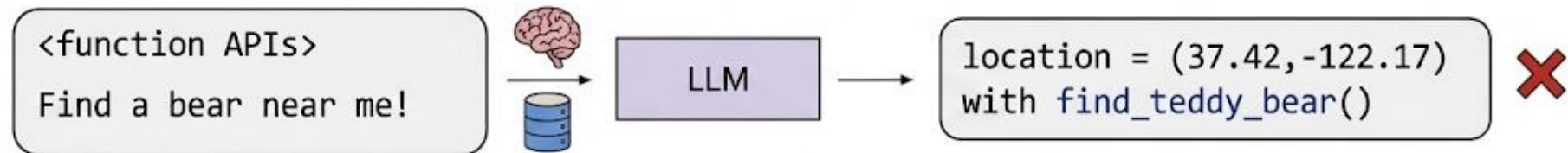
{"name": "Teddy", ...}

LLM

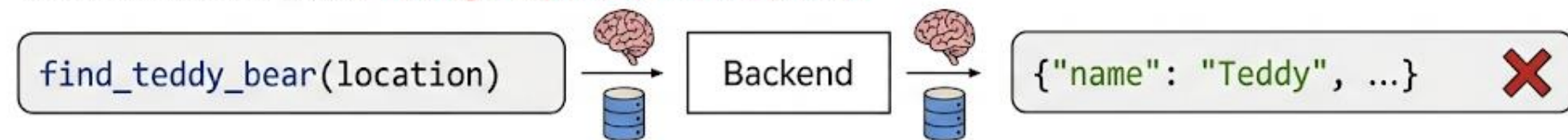
response

Possible Failures

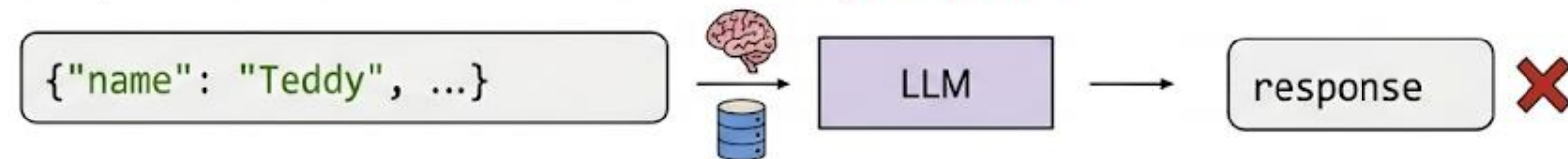
Tool Prediction Failures: does not use/hallucinates/uses the wrong tool, infers wrong argument



Tool Call Failures: wrong response, no response



Response Generation Failures: wrong response



Comprehensive LLM Tool Call Failure Categories: Analysis & Remedies

Failure Category	Symptom	Potential Causes & Remedies
 Tool Prediction Error: Does Not Use Tool	LLM directly issues a response.	- Tool router error - Model doesn't know how to use the tool - Retrain the tool router - SFT-train the model / adjust prompt
 Tool Prediction Error: Hallucinates Tool	LLM uses a tool that doesn't exist.	- Model is too weak - API naming is not logical - Instructions are unclear - Upgrade model - Revamp API - Iterate instructions
 Tool Prediction Error: Uses Wrong Tool	LLM uses another available tool.	- Tool router error - Model chose to use the wrong tool - Retrain tool router - SFT-train model / adjust prompt
 Tool Prediction Error: Infers Wrong Argument	LLM doesn't feed the right argument to the tool.	- Argument cannot be inferred - Model doesn't know how to use the tool - Introduce helper tool / context - SFT-train model / adjust prompt
 Tool Call Error: Wrong Response	Tool returns wrong response or an error.	- Tool returns incorrect value - Tool errors instead of value FIX THE TOOL IMPLEMENTATION! * Errors can be legitimate but non-standard practice
 Tool Call Error: No Response	The tool doesn't return anything.	- Bug or "feature" of implementation? - Return something (even empty JSON) - Emit meaningful tool outputs
 Response Generation Error: Wrong Response	Final response doesn't convey the tool's response.	- Model lacks grounding - Tool response spams context window - Tool output is not meaningful - Upgrade response synthesis LLM - Trim information - Make output format descriptive

LLMs Can Teach Themselves to Use Tools

- We can use SFT to train LLMs to utilize the tools.

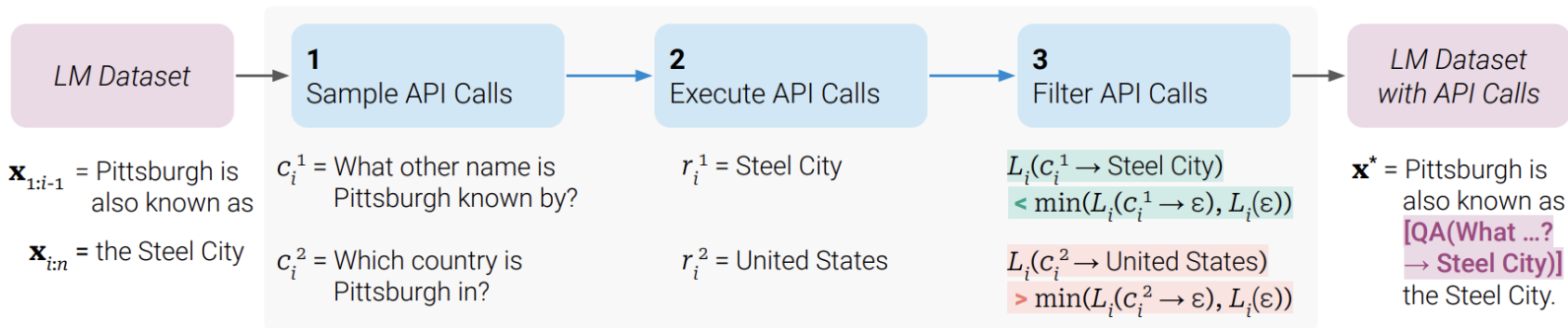


Figure 2: Key steps in our approach, illustrated for a *question answering* tool: Given an input text $\mathbf{x}$, we first sample a position i and corresponding API call candidates $c_i^1, c_i^2, \dots, c_i^k$. We then execute these API calls and filter out all calls which do not reduce the loss L_i over the next tokens. All remaining API calls are interleaved with the original text, resulting in a new text $\mathbf{x}^*$.

Multi-Turn Dialogue Algorithm

Formal Multi-Turn Reward:

For conversation $C = \{(u_1, r_1), \dots, (u_t, r_t)\}$:

Terminal reward:

$$R_{\text{term}}(C) = \begin{cases} +10 & \text{task completed} \\ +5 & \text{partial progress} \\ 0 & \text{no progress} \\ -5 & \text{task failed} \end{cases}$$

Per-turn rewards:

$$r_t = r_{\text{quality}}(r_t) + r_{\text{progress}}(C_{1:t})$$

Total reward:

$$R_{\text{total}}(C) = \sum_{t=1}^T \gamma^{T-t} r_t + R_{\text{term}}(C)$$

where γ is discount factor (typically 0.95-0.99).

Earlier turns get discounted more (focus on recent quality).

Plain English:

Conversation reward has two parts:

1. Per-turn rewards:

- Quality of each response
- Progress toward goal 🎯
- User engagement

Each turn gets a small reward based on local quality.

2. Terminal reward:

- Did conversation achieve goal?
- Large reward at end if successful

Example: Tech support conversation

Turn 1: "What's the problem?"

→ **Local reward:** +1 (good question)

Turn 2: "Have you tried restarting?"

→ **Local reward:** +2 (standard diagnostic)

Turn 3: "That fixed it!"

→ **Terminal reward:** +10 (problem solved!)

Total: +13 🍀

Model learns: asking good diagnostic questions early leads to successful resolution!

Breaking It Down Step-by-Step

Scenario: User needs help with password reset

Conversation 1 (Poor)

Turn	Exchange	Reward
1	User: "I can't log in" Bot: "Have you tried using Incognito mode?"	— +1
2	User: "That doesn't help" Bot: "Clear your cookies"	— +1
3	User: "Still can't log in!" Bot: "Try a different browser"	— 0
4	User: "This isn't working. Bye."; (Task failed)	— -5
Total reward:		-3

Bot never diagnosed the actual problem (forgotten password)!

Conversation 2 (Good)

Turn	Exchange	Reward
1	User: "I can't log in" Bot: "Are you getting an error message?"	— +2
2	User: "It says 'incorrect password'" Bot: "Let me help reset your password..."	+3
3	User: "Got the reset email, thanks!" Bot: "Great! Let me know if you need anything else." (Task completed)	+10
Total reward:		+17

Bot quickly diagnosed and solved the problem!

What the model learns:

-  Diagnostic questions early → higher terminal reward
-  Random suggestions without diagnosis → low reward
-  Quick resolution → bonus reward
-  User satisfaction signals ("thanks!") → positive signal



THANK YOU

The image features a dark, futuristic background with a glowing blue brain at the top center and a glowing blue robotic hand at the bottom center, pointing upwards towards the brain. The text "THANK YOU" is displayed in a large, stylized, glowing blue font in the center. The background is filled with various digital elements, including line graphs, bar charts, and hexagonal patterns, all rendered in a glowing blue color. The overall aesthetic is high-tech and digital.